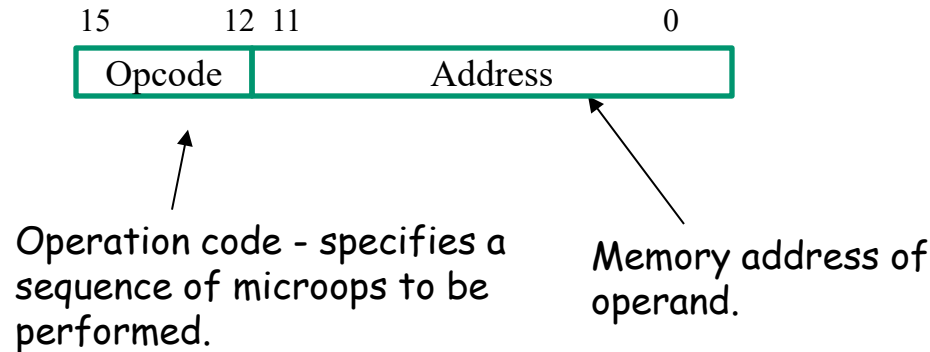
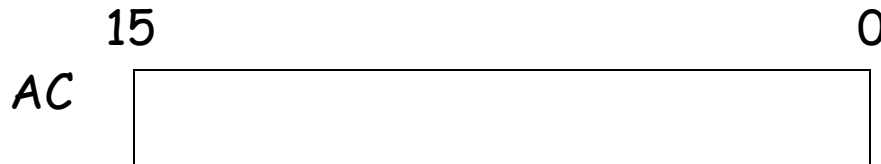


Chapter 5: Basic Computer Organization and Design

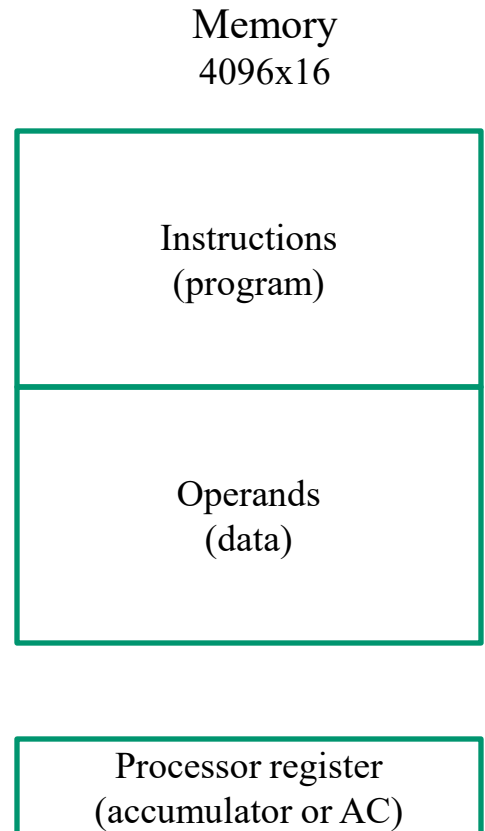
Very simple instruction code format.



Our computer has one processor register, called the accumulator, *AC*



Memory is used to hold the sequence of instructions (the program) and the operands (data).



Three types of addressing

Effective address

Instruction format

Memory

Operand

AC

Direct address

Indirect address

CS 2613

Computer Instructions

Basic computer instruction formats.

I=0: Direct Address
I=1: Indirect Address

(a) Memory-reference instruction

(b) Register-reference instruction

(c) Input-output instruction

CS 2613

Control Unit

Control Unit:

Two possible types of control:

- hardwired (assumed in Chapter 5)
- microprogrammed (covered in a later chapter)

From Figure 5-6, note that counter may not always count to 15, because it may be cleared.

CS 2613

Instruction Cycle (four subcycles)

1. **Fetch** an instruction from memory
2. **Decode** the instruction
3. **Read** the effective address from memory (if indirect addressing is used)
4. **Execute** the instruction.

CS 2613

Register Instructions: Operations during T_2

TABLE 5-3 Execution of Register-Reference Instructions

CS 2613

M-R Instructions: Operations during T_4 onwards

Note: Be careful when looking at Table 5-4 p. 145. Those are not technically RTL statements because they take more than one clock cycle.

AND

$D_0T_4: DR \leftarrow M[AR]$

$D_0T_5: AC \leftarrow AC \wedge DR, SC \leftarrow 0$

Note: There is an implied, "SC ← SC + 1" here.

CS 2613

I/O Instructions

I/O Device - a terminal with a keyboard (for input) and printer (for output).

- INPR and OUTR each hold eight bits
- Associated with INPR is a 1-bit flag, FGI.

When a key is struck on the keyboard, the 8-bit code is shifted into INPR, and FGI is set to 1 by the input device.

- FGI=1 means data is in INPR and is ready to be read.
- After transferring INPR into AC, computer should clear FGI.

When the output device is ready to receive an output character, it sets FGO to 1.

After loading data into OUTR, the computer clears FGO, and does not send more data until output device sets FGO back to 1.

CS 2613

Interrupts

Note: In some applications, polling is necessary because there may not be other processing that can be done until input (or output) is performed.

However, in some cases interrupt control is more efficient (the I/O device interrupts the computer to let it know a transfer is ready).

When the IEN flag is set to 1, then the flags FGI and FGO interrupt the computer.

Flags:

1. FGI
2. FGO
3. IEN

CS 2613

Design of Control Logic Gates (of Fig 5-16)

Control Logic Gates

Inputs:

- 2 decoders
- 1 flip flop
- Bits 0-11 of IR
- Bits 0-15 of AC to check if AC=0 and sign (AC(15))
- Bits 0-15 of DR to check if DR=0
- 7 flip flops

Outputs: Signals to control

- 9 registers
- R/W of memory
- Set, clear, complement FFs
- S_0, S_1, S_2 to select a register for bus
- AC adder and logic circuit

CS 2613

Effective
address



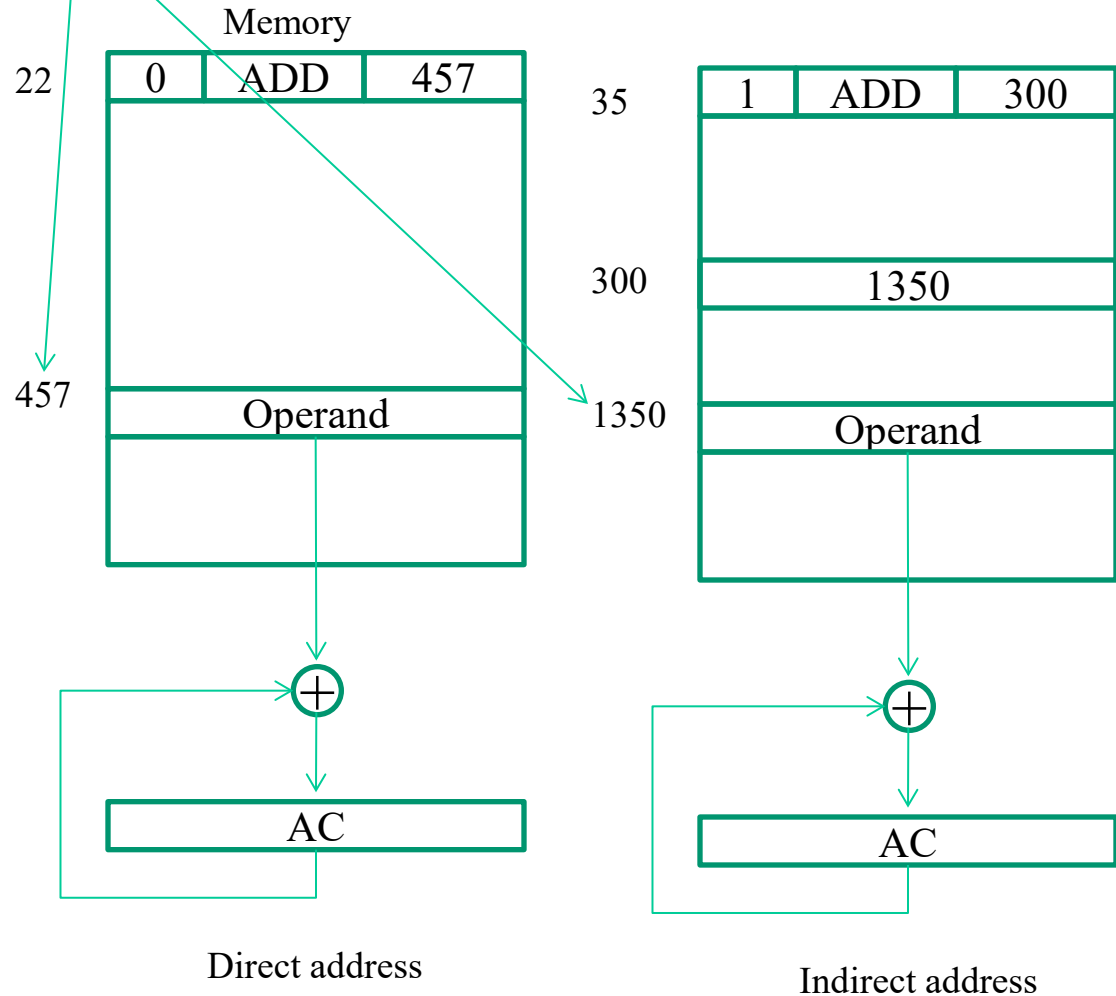
Instruction format

Three types of addressing

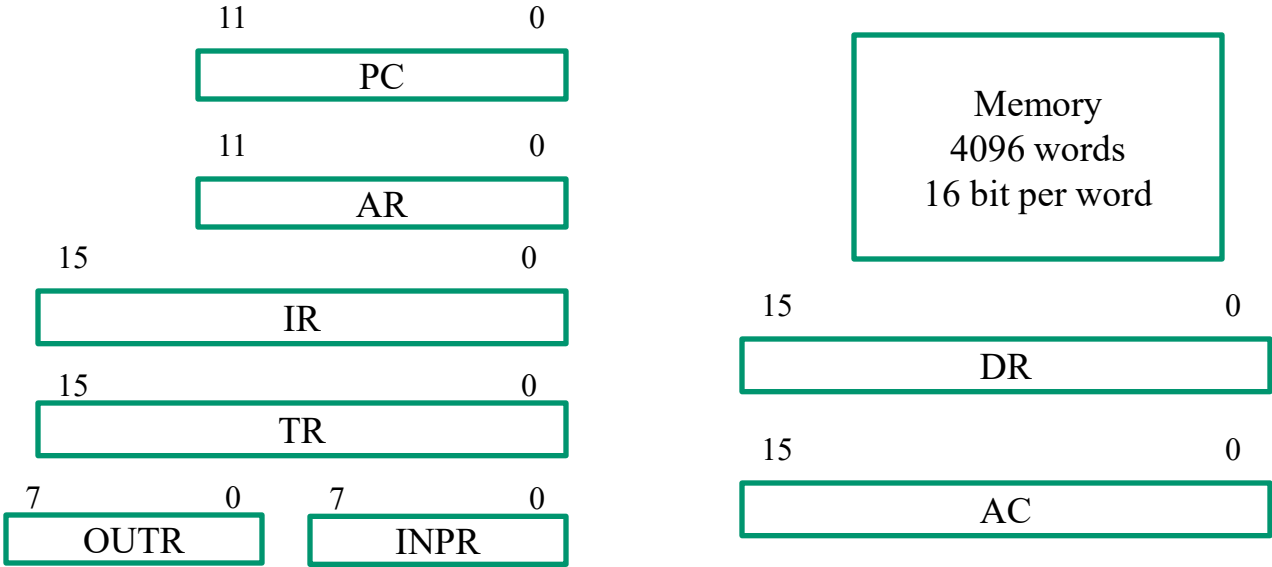
1. *immediate operand*: the 2nd part of the instruction (bits 0-11) is actually the operand data (not used in this chapter)

2. *direct addressing*: 2nd part of instruction specifies the address of the operand.

3. *indirect addressing*: 2nd part of instruction specifies the address in the memory in which the address of the operand is stored.

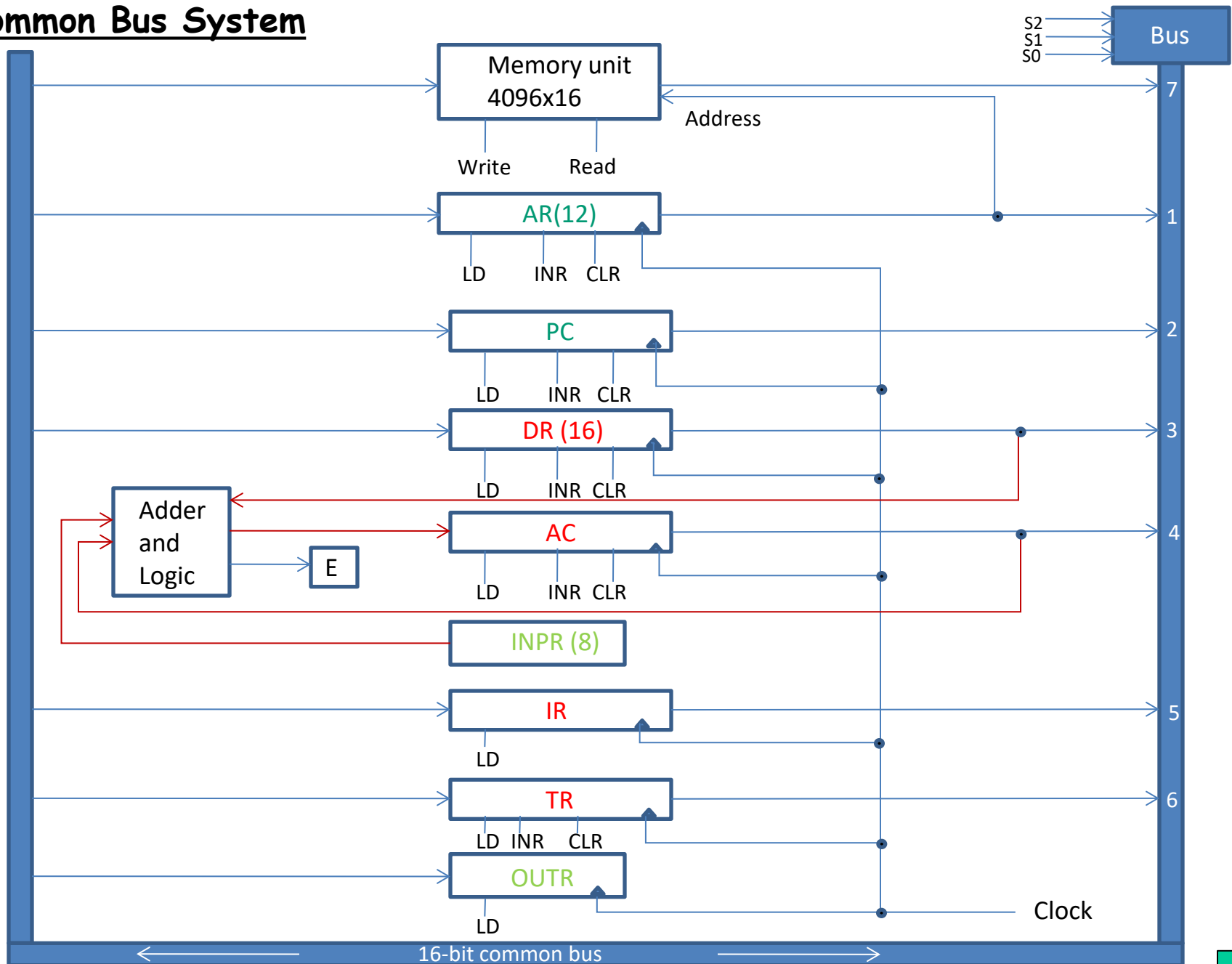


Computer Registers



Register symbol	Number Of bits	Register name	Function
DR	16	Data register	Holds memory operand
AR	12	Address register	Holds address for memory
AC	16	Accumulator	Processor register
IR	16	Instruction register	Holds instruction code
PC	12	Program counter	Holds address of instruction
TR	16	Temporary register	Holds temporary data
INPR	8	Input register	Holds input character
OUTR	8	Output register	Holds output character

Common Bus System



Notes:

1. The bus could be implemented with MUXs or tri-state buffers.
2. The LD (load), INR, and CLR lines of the registers are independent (they are not tied together).
3. A "write" to the memory unit is analogous to loading a register.
4. DR and AC are used for arithmetic operations.
5. Any register can receive data from memory after a read, except AC and INPR.
6. The E bit is the "extended" AC bit (e.g., used for carry-out for addition).
7. AR and PC are only 12 bits. Why?

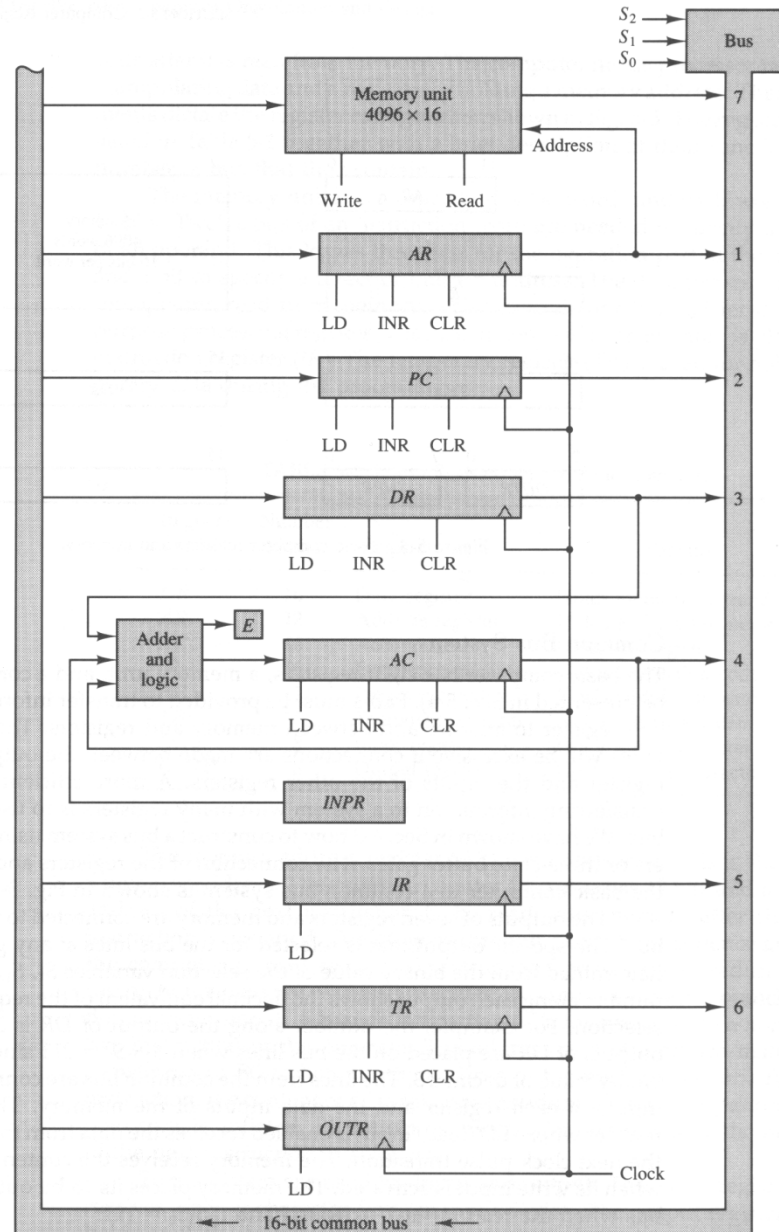


Figure 5-4 Basic computer registers connected to a common bus.



- 5-3. The following control inputs are active in the bus system shown in Fig. 5-4. For each case, specify the register transfer that will be executed during the next clock transition.

	S_2	S_1	S_0	LD of register	Memory	Adder
a.	1	1	1	<i>IR</i>	Read	—
b.	1	1	0	<i>PC</i>	—	—
c.	1	0	0	<i>DR</i>	Write	—
d.	0	0	0	<i>AC</i>	—	Add

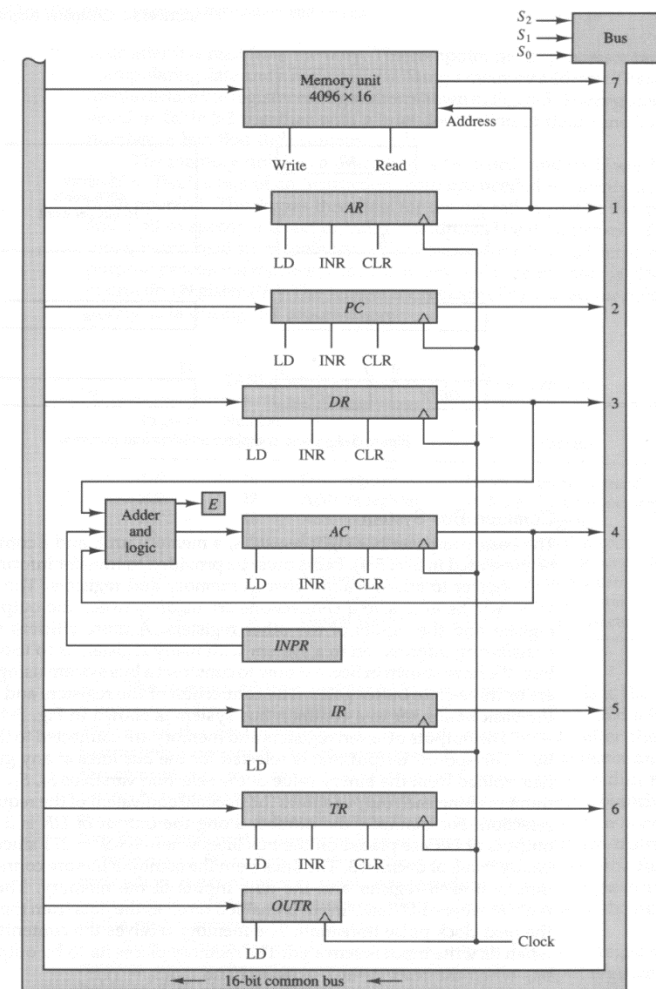
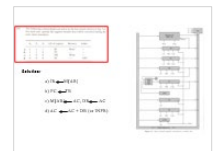


Figure 5-4 Basic computer registers connected to a common bus.



- 5-5. Explain why each of the following microoperations cannot be executed during a single clock pulse in the system shown in Fig. 5-4. Specify a sequence of microoperations that will perform the operation.
- $IR \leftarrow M[PC]$
 - $AC \leftarrow AC + TR$
 - $DR \leftarrow DR + AC$ (AC does not change)

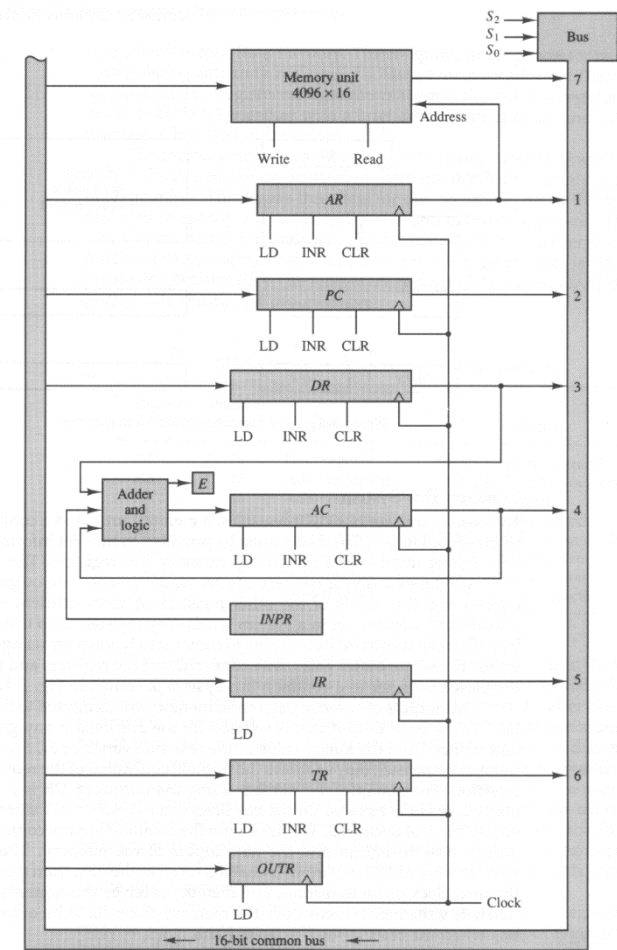


Figure 5-4 Basic computer registers connected to a common bus.

Computer Instructions

Basic computer instruction formats.

I=0: Direct Address
I=1: Indirect Address



(a) Memory-reference instruction



(b) Register-reference instruction

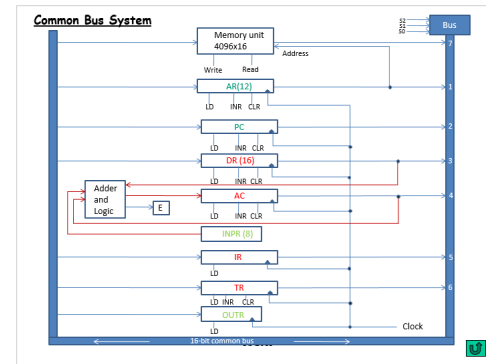


(c) Input-output instruction

(Opcode=000 through 110)

(Opcode=111, I=0)

(Opcode=111, I=1)



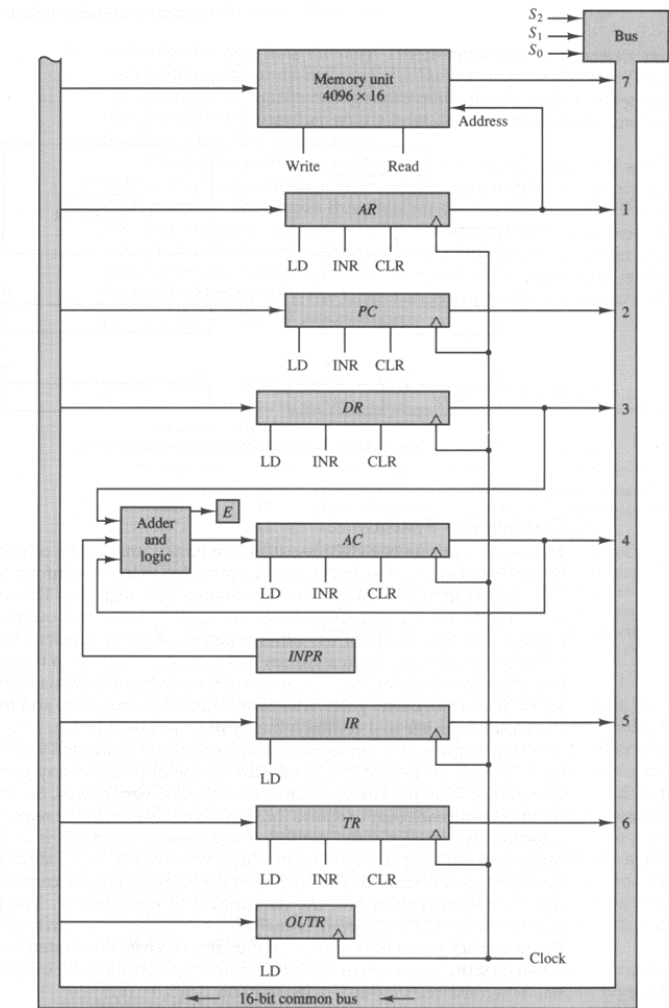
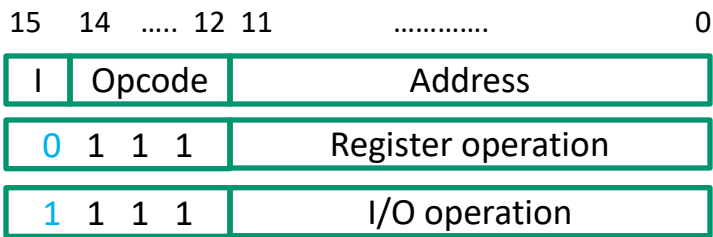


Figure 5-4 Basic computer registers connected to a common bus.

Symbol	Hexadecimal code		Description
	I=0	I=1	
AND	0xxx	8xxx	AND memory word to AC
ADD	1xxx	9xxx	Add memory word to AC
LDA	2xxx	Axxx	Load memory word to AC
STA	3xxx	Bxxx	Store content of AC in memory
BUN	4xxx	Cxxx	Branch unconditionally
BSA	5xxx	Dxxx	Branch and save return address
ISZ	6xxx	Exxx	Increment and skip if zero
CLA	7800		Clear AC
CLE	7400		Clear E
CMA	7200		Complement AC
CME	7100		Complement E
CIR	7080		Circulate right AC and E
CIL	7040		Circulate left AC and E
INC	7020		Increment AC
SPA	7010		Skip next instruction if AC positive
INP	F800		Input character to AC
OUT	F400		Output character from AC
SKI	F200		Skip on input flag
SKO	F100		Skip on output flag



Q: How to decode the register-reference instructions.

Q: How to decode the I/O instructions.

A: Look closely at Table 5-2.

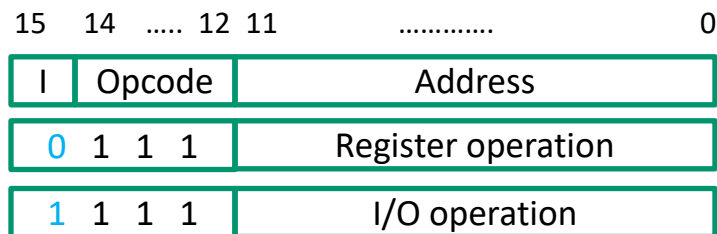


TABLE 5-2 Basic Computer Instructions

Symbol	Hexadecimal code		Description
	I = 0	I = 1	
AND	0xxx	8xxx	AND memory word to AC
ADD	1xxx	9xxx	Add memory word to AC
LDA	2xxx	Axxx	Load memory word to AC
STA	3xxx	Bxxx	Store content of AC in memory
BUN	4xxx	Cxxx	Branch unconditionally
BSA	5xxx	Dxxx	Branch and save return address
ISZ	6xxx	Exxx	Increment and skip if zero
CLA	7800		Clear AC
CLE	7400		Clear E
CMA	7200		Complement AC
CME	7100		Complement E
CIR	7080		Circulate right AC and E
CIL	7040		Circulate left AC and E
INC	7020		Increment AC
SPA	7010		Skip next instruction if AC positive
SNA	7008		Skip next instruction if AC negative
SZA	7004		Skip next instruction if AC zero
SZE	7002		Skip next instruction if E is 0
HLT	7001		Halt computer
INP	F800		Input character to AC
OUT	F400		Output character from AC
SKI	F200		Skip on input flag
SKO	F100		Skip on output flag
ION	F080		Interrupt on
IOF	F040		Interrupt off

BUN and BSA

$T_0: AR \leftarrow PC$
 $T_1: DR \leftarrow R[AR]; NG \leftarrow NGI$
 $T_2: DR_1 \leftarrow DR + \text{immed}(DR_1, DR_1, AR) \leftarrow DR(DR_1); I \leftarrow DR_1(NG)$
 $[I \neq T_0: \text{Nothing (NOP)}]$
 $[I \neq T_1: AR \leftarrow R[AR]]$

BUN: (branch unconditionally)

Recall that indirect addressing could've been used.

$D_4T_4: PC \leftarrow AR, \quad SC \leftarrow 0$
 (Recall PC was already incremented at T_1 .)

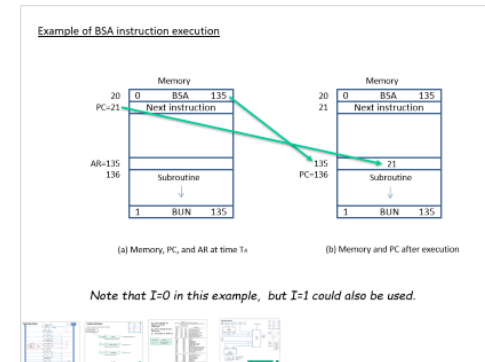
BSA: (Branch and Save Return Address)

$D_5T_4: M[AR] \leftarrow PC, \quad AR \leftarrow AR + 1$
 $D_5T_5: PC \leftarrow AR, \quad SC \leftarrow 0$

Similar to
Function Call





5-1. A computer uses a memory unit with 256K words of 32 bits each. A binary instruction code is stored in one word of memory. The instruction has four parts: an indirect bit, an operation code, a register code part to specify one of 64 registers, and an address part.

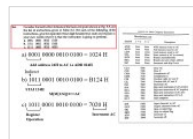
- How many bits are there in the operation code, the register code part, and the address part?
- Draw the instruction word format and indicate the number of bits in each part.
- How many bits are there in the data and address inputs of the memory?

5-6. Consider the instruction formats of the basic computer shown in Fig. 5-5 and the list of instructions given in Table 5-2. For each of the following 16-bit instructions, give the equivalent four-digit hexadecimal code and explain in your own words what it is that the instruction is going to perform.

- a. 0001 0000 0010 0100
- b. 1011 0001 0010 0100
- c. 0111 0000 0010 0000

TABLE 5-2 Basic Computer Instructions

Symbol	Hexadecimal code		Description
	I = 0	I = 1	
AND	0xxx	8xxx	AND memory word to AC
ADD	1xxx	9xxx	Add memory word to AC
LDA	2xxx	Axxx	Load memory word to AC
STA	3xxx	Bxxx	Store content of AC in memory
BUN	4xxx	Cxxx	Branch unconditionally
BSA	5xxx	Dxxx	Branch and save return address
ISZ	6xxx	Exxx	Increment and skip if zero
CLA	7800		Clear AC
CLE	7400		Clear E
CMA	7200		Complement AC
CME	7100		Complement E
CIR	7080		Circulate right AC and E
CIL	7040		Circulate left AC and E
INC	7020		Increment AC
SPA	7010		Skip next instruction if AC positive
SNA	7008		Skip next instruction if AC negative
SZA	7004		Skip next instruction if AC zero
SZE	7002		Skip next instruction if E is 0
HLT	7001		Halt computer
INP	F800		Input character to AC
OUT	F400		Output character from AC
SKI	F200		Skip on input flag
SKO	F100		Skip on output flag
ION	F080		Interrupt on
IOF	F040		Interrupt off



I/O Instructions

I/O Device - a terminal with a keyboard (for input) and printer (for output).

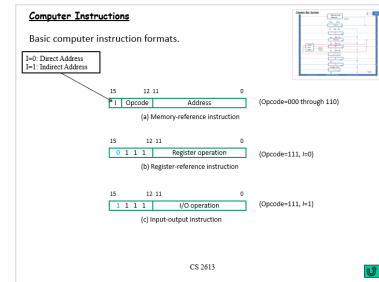
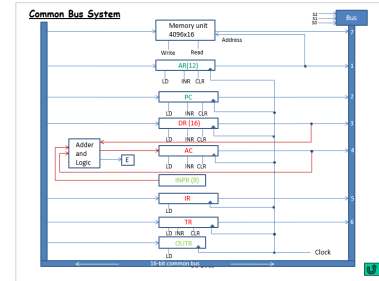
- INPR and OUTF each hold eight bits
- Associated with INPR is a 1-bit flag, FGI.

When a key is struck on the keyboard, the 8-bit code is shifted into INPR, and FGI is set to 1 by the input device.

- **FGI=1 means data is in INPR** and is ready to be read.
- After transferring INPR into AC, computer should clear FGI.

When the output device is ready to receive an output character, it sets FGO to 1.

After loading data into OUTF, the computer clears FGO, and does not send more data until **output device sets FGO back to 1**.

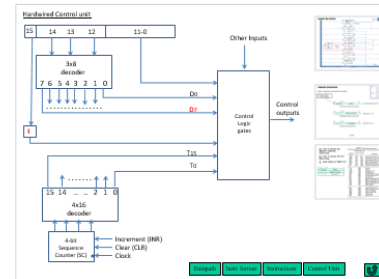


Q: How to decode the register-reference instructions.

Q: How to decode the I/O instructions.

A: Look closely at Table 5-2.

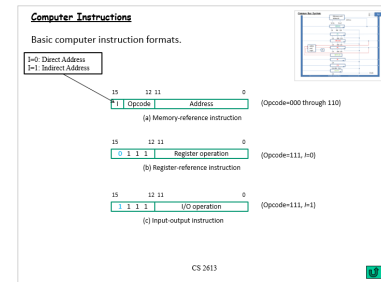
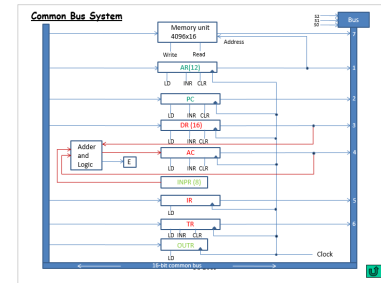
Symbol	Hexadecimal code	Description
AND	0000	AND memory word to AC
ADD	0001	Add memory word to AC
LDA	0010	Load memory word to AC
STA	0011	Store content of AC in memory
BCR	0100	Branch conditionally
BRA	0101	Branch and save return address
DEC	0110	Increment and skip if zero
CLA	1000	Clear AC
CLE	1001	Clear E
CMA	1010	Complement AC
CME	1011	Complement E
CIR	1100	Circular right AC and E
CIL	1101	Circular left AC and E
INC	1110	Increment AC
INC	1111	Increment E
SFA	0010	Skip next instruction if AC positive
SNA	0011	Skip next instruction if AC negative
SZA	0100	Skip next instruction if Z is 0
SZE	0101	Skip next instruction if E is 0
HLT	0000	Halting instruction
INP	1000	Input character to AC
OUT	1001	Output character from AC
SKI	1010	Skip on input flag
SNO	1011	Skip on output flag
ION	1100	Interrupt on
IOF	1101	Interrupt off



Input Output Instructions

D7IT3 = p (common to all input output instructions)
IR(i) = Bi[bit in IR(6-11) that specifies the operation]

	p:	SC $\leftarrow$ 0	Clear SC
INP	pB11	AC(0-7) $\leftarrow$ INPR, FGI $\leftarrow$ 0	Input character
OUT	pB10	OUTR $\leftarrow$ AC(0-7), FGO $\leftarrow$ 0	Output character
SKI	pB9	If(FGI=1)then (PC $\leftarrow$ PC+1)	Skip on input flag
SKO	pB8	If(FGO=1)then (PC $\leftarrow$ PC+1)	Skip on output flag
ION	pB7	IEN $\leftarrow$ 1	Interrupt enable on
IOF	pB6	IEN $\leftarrow$ 0	Interrupt enable off

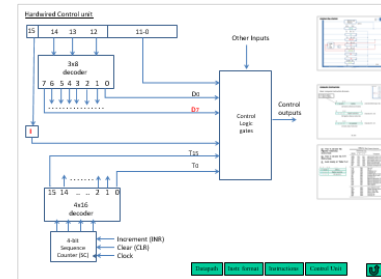


Q: How to decode the register-reference instructions.

A: Look to decode the I/O instructions.

A: Look closely at Table 5-2.

Hexadecimal code		Description	
Symbol	$I = 1$	$I = 0$	
AND	AND	AND memory word to AC	
ASR	ASR	Asl memory word to AC	
ADD	ADD	Add memory word to AC	
ADSD	ADSD	Add signed AC to AC	
ASR	ASR	Asl signed AC to AC	
STA	STA	Store AC to memory	
STAD	STAD	Store signed AC to memory	
STB	STB	Store byte and save return address	
STBD	STBD	Store signed byte and save return address	
STC	STC	Store AC and step 2 return	
CLA	7000	Clear AC	
CLE	7004	Clear E	
CLC	7008	Clear C	
COM	700C	Complement AC	
CMO	7100	Complement E	
CMR	7104	Complement C	
CSA	7200	Circular shift AC left and E	
CSL	7204	Circular shift left and C	
ISC	7208	Invert AC	
STA	7010	Step next instruction if AC positive	
STA	7014	Step next instruction if AC negative	
STA	7018	Step next instruction if E zero	
STA	701C	Step next instruction if E zero	
HLT	7001	Halt computer	
INP	8000	Input character to AC	
OUT	8004	Output character from AC	
SKI	F200	Step on input flag	
SKN	F204	Step on negative flag	
ISN	F208	Interrupt on	
IOF	F20C	Interrupt off	



Typical programmed control transfer (polling)

10		SKI
11		BUN 10
12		INP
.....		
27		SKO
28		BUN 27
29		OUT

CS 2613

- 5-10.** An instruction at address 021 in the basic computer has $I = 0$, an operation code of the AND instruction, and an address part equal to 083 (all numbers are in hexadecimal). The memory word at address 083 contains the operand B8F2 and the content of AC is A937. Go over the instruction cycle and determine the contents of the following registers at the end of the execute phase: PC, AR, DR, AC, and IR. Repeat the problem six more times starting with an operation code of another memory-reference instruction.

5-10. An instruction at address 021 in the basic computer has $I = 0$, an operation code of the AND instruction, and an address part equal to 083 (all numbers are in hexadecimal). The memory word at address 083 contains the operand B8F2 and the content of AC is A937. Go over the instruction cycle and determine the contents of the following registers at the end of the execute phase: PC, AR, DR, AC, and IR. Repeat the problem six more times starting with an operation code of another memory-reference instruction.

PC	AR	DR	AC	IR
021	000	000	A937	00000000
021	083	000	A937	00000000
021	083	B8F2	A937	00000000
021	083	B8F2	0000	00000000
021	083	B8F2	0000	00000000
021	083	B8F2	0000	00000000
021	083	B8F2	0000	00000000
021	083	B8F2	0000	00000000

Control Unit:

Two possible types of control:

- hardwired (assumed in Chapter 5)
- microprogrammed
(covered in a later chapter)

From Figure 5-6, note that counter may not always count to 15, because it may be cleared.

Control Unit

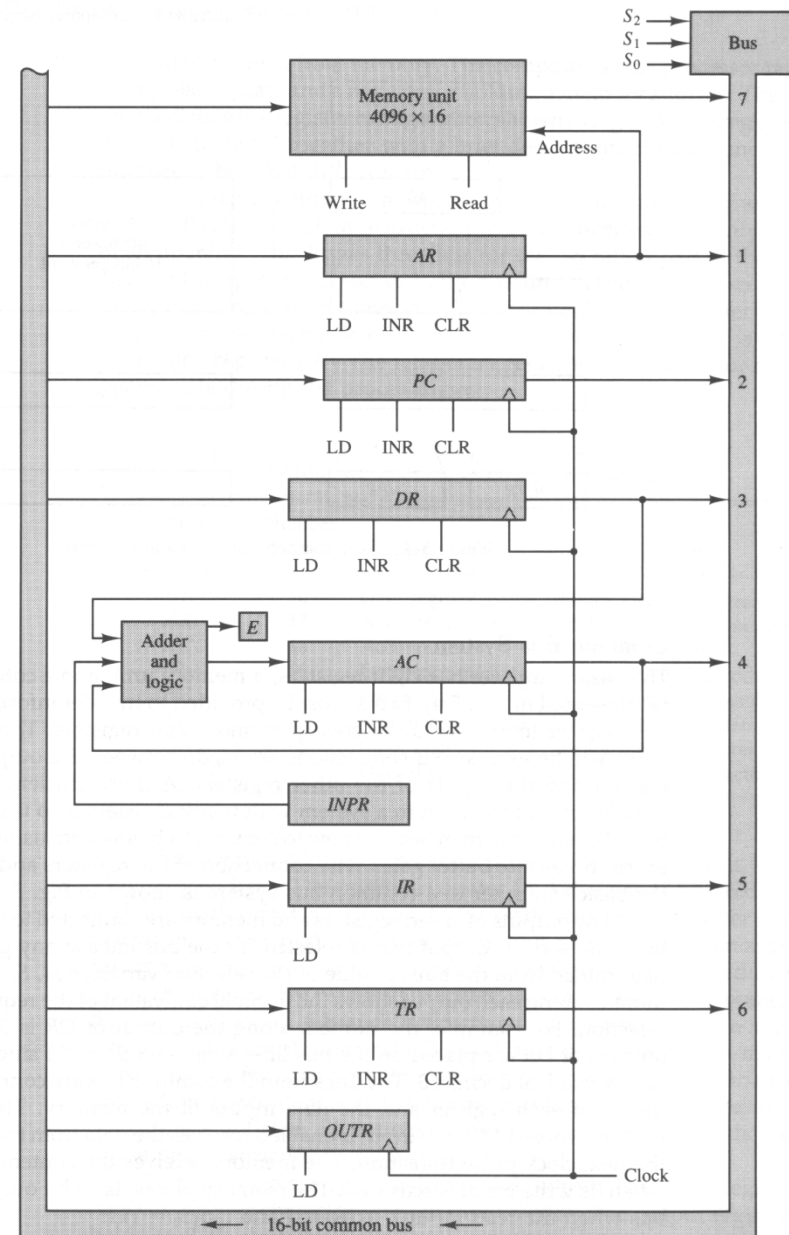
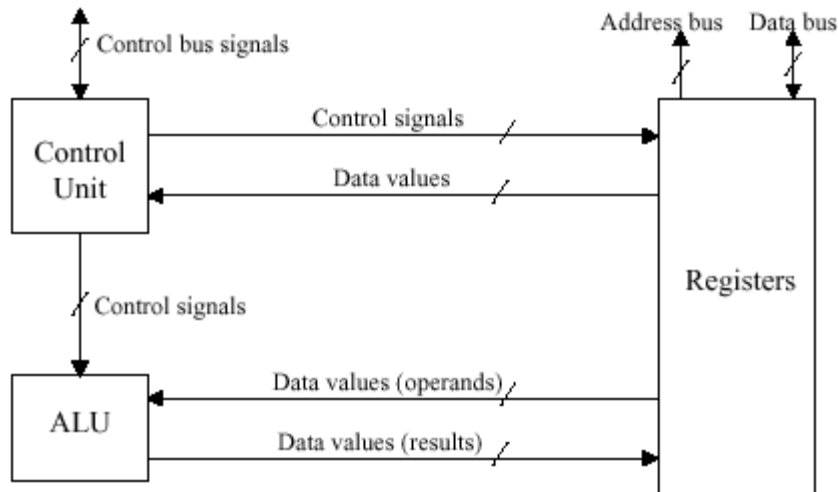


Figure 5-4 Basic computer registers connected to a common bus.

Hardwired Control unit

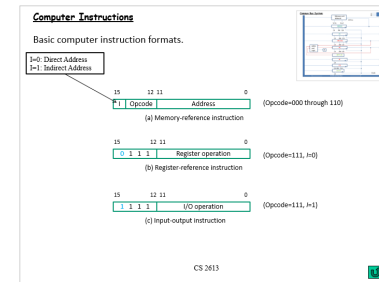
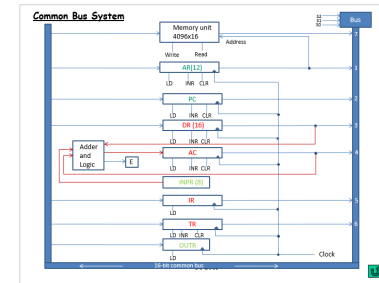
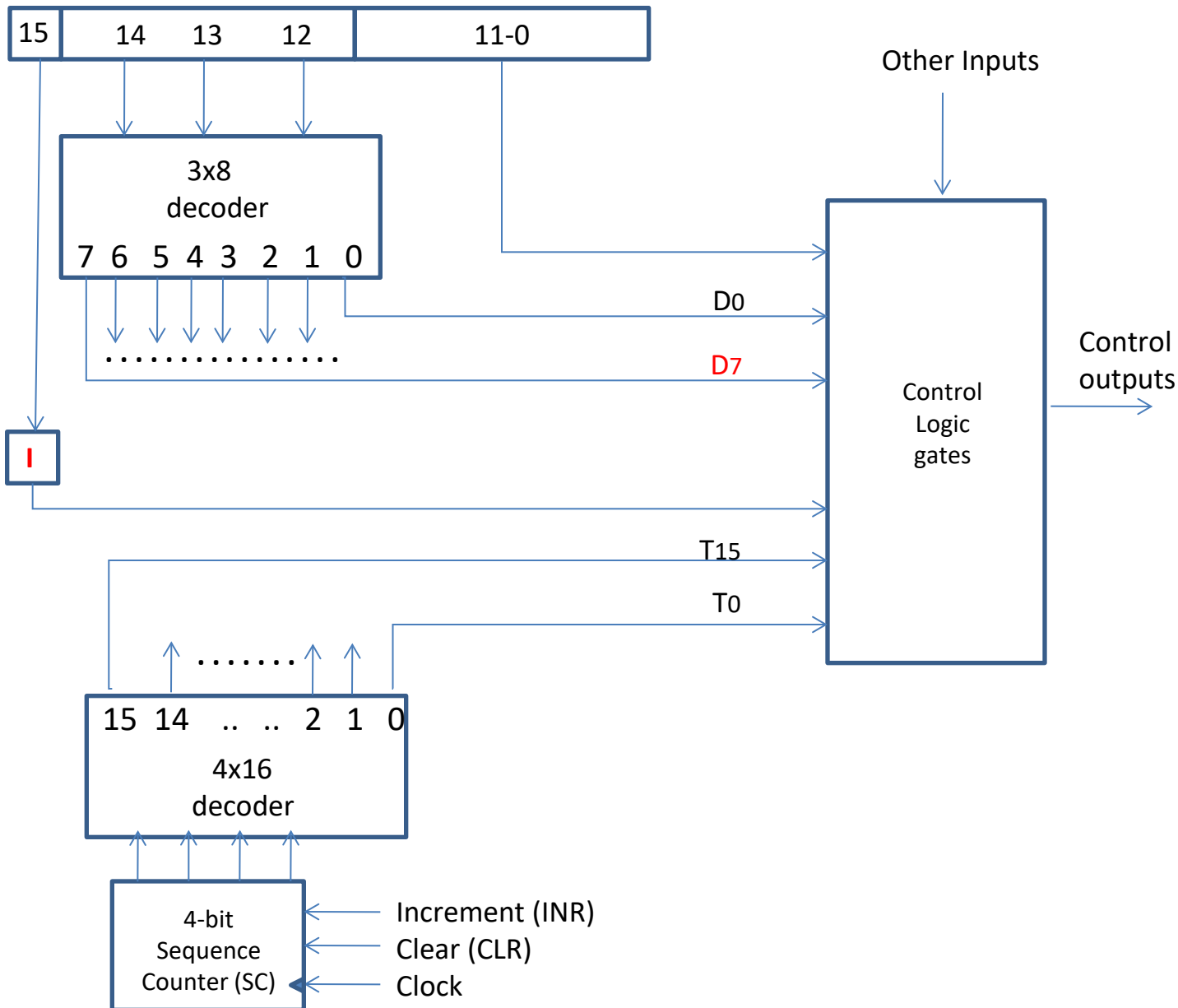
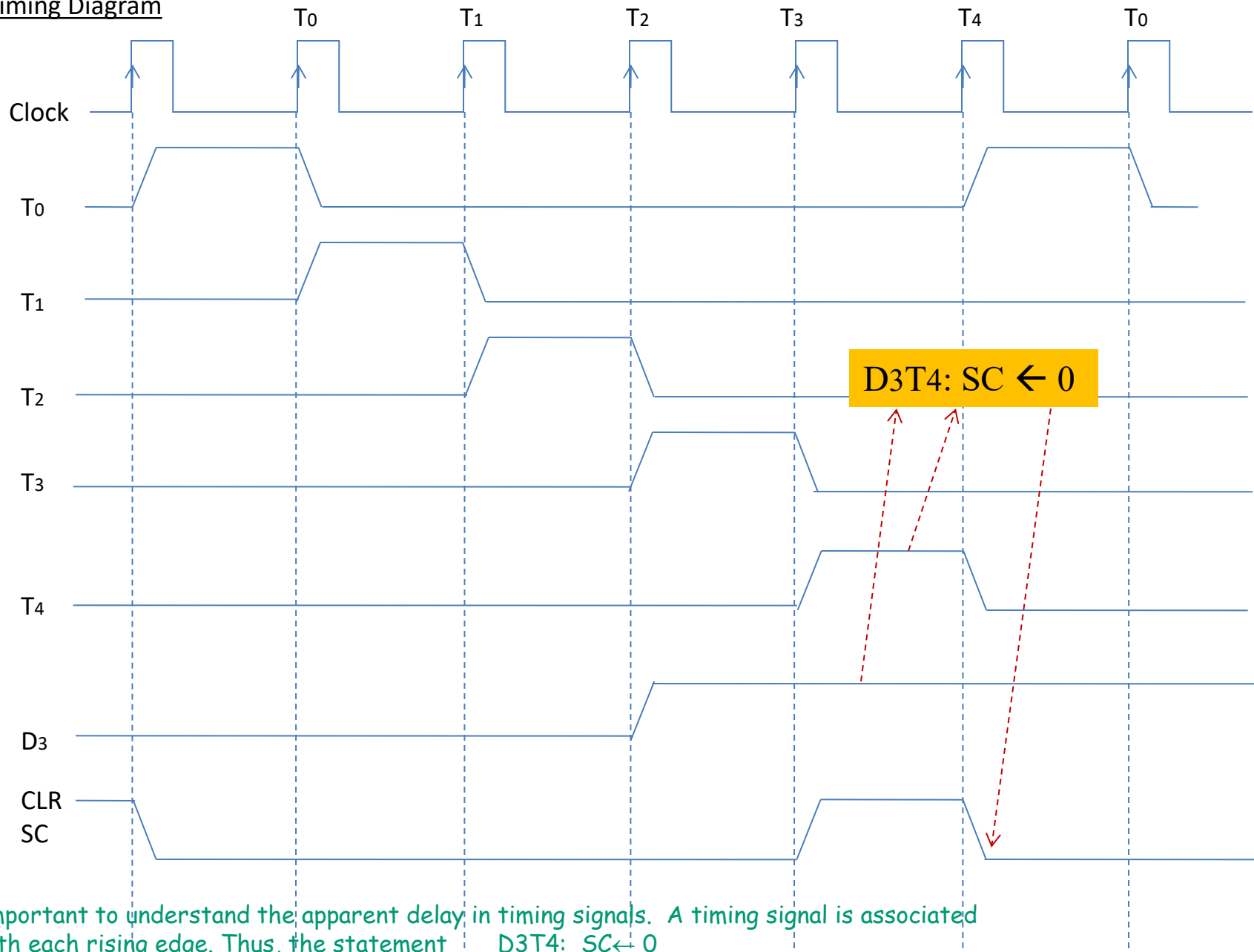


TABLE 5-2 Basic Computer Instructions			
Hexadecimal code		$i = 0 \text{ or } 1$	Description
Symbol			
AND	AND	AND	AND memory word at AC
AND	AND	AND	Add memory word at AC
AND	AND	AND	AND memory word at AC
ASL	ASL	ASL	Shift memory word at AC
STA	STA	STA	Store memory word at AC
STB	STB	STB	Store memory word at B memory
STC	STC	STC	Store memory word at C memory
BSA	BSA	BSA	Branch and save return address
BSE	BSE	BSE	Branch and save step return
CLA	7400	CLA	Clear AC
CMA	7401	CMA	Clear C
CMB	7402	CMB	Clear B
CME	7403	CME	Clear E
CMV	7404	CMV	Clear V
CMZ	7405	CMZ	Clear Z
CMF	7406	CMF	Clear F
CMH	7407	CMH	Clear H
CMR	7408	CMR	Clear R
CMY	7409	CMY	Clear Y
CMX	740A	CMX	Clear X
CMZ	740B	CMZ	Clear Z
CMF	740C	CMF	Clear F
CMH	740D	CMH	Clear H
CMR	740E	CMR	Clear R
CMY	740F	CMY	Clear Y
CMX	7410	CMX	Clear X
CMZ	7411	CMZ	Clear Z
CMF	7412	CMF	Clear F
CMH	7413	CMH	Clear H
CMR	7414	CMR	Clear R
CMY	7415	CMY	Clear Y
CMX	7416	CMX	Clear X
CMZ	7417	CMZ	Clear Z
CMF	7418	CMF	Clear F
CMH	7419	CMH	Clear H
CMR	741A	CMR	Clear R
CMY	741B	CMY	Clear Y
CMX	741C	CMX	Clear X
CMZ	741D	CMZ	Clear Z
CMF	741E	CMF	Clear F
CMH	741F	CMH	Clear H
CMR	7420	CMR	Clear R
CMY	7421	CMY	Clear Y
CMX	7422	CMX	Clear X
CMZ	7423	CMZ	Clear Z
CMF	7424	CMF	Clear F
CMH	7425	CMH	Clear H
CMR	7426	CMR	Clear R
CMY	7427	CMY	Clear Y
CMX	7428	CMX	Clear X
CMZ	7429	CMZ	Clear Z
CMF	742A	CMF	Clear F
CMH	742B	CMH	Clear H
CMR	742C	CMR	Clear R
CMY	742D	CMY	Clear Y
CMX	742E	CMX	Clear X
CMZ	742F	CMZ	Clear Z
CMF	7430	CMF	Clear F
CMH	7431	CMH	Clear H
CMR	7432	CMR	Clear R
CMY	7433	CMY	Clear Y
CMX	7434	CMX	Clear X
CMZ	7435	CMZ	Clear Z
CMF	7436	CMF	Clear F
CMH	7437	CMH	Clear H
CMR	7438	CMR	Clear R
CMY	7439	CMY	Clear Y
CMX	743A	CMX	Clear X
CMZ	743B	CMZ	Clear Z
CMF	743C	CMF	Clear F
CMH	743D	CMH	Clear H
CMR	743E	CMR	Clear R
CMY	743F	CMY	Clear Y
CMX	7440	CMX	Clear X
CMZ	7441	CMZ	Clear Z
CMF	7442	CMF	Clear F
CMH	7443	CMH	Clear H
CMR	7444	CMR	Clear R
CMY	7445	CMY	Clear Y
CMX	7446	CMX	Clear X
CMZ	7447	CMZ	Clear Z
CMF	7448	CMF	Clear F
CMH	7449	CMH	Clear H
CMR	744A	CMR	Clear R
CMY	744B	CMY	Clear Y
CMX	744C	CMX	Clear X
CMZ	744D	CMZ	Clear Z
CMF	744E	CMF	Clear F
CMH	744F	CMH	Clear H
CMR	7450	CMR	Clear R
CMY	7451	CMY	Clear Y
CMX	7452	CMX	Clear X
CMZ	7453	CMZ	Clear Z
CMF	7454	CMF	Clear F
CMH	7455	CMH	Clear H
CMR	7456	CMR	Clear R
CMY	7457	CMY	Clear Y
CMX	7458	CMX	Clear X
CMZ	7459	CMZ	Clear Z
CMF	745A	CMF	Clear F
CMH	745B	CMH	Clear H
CMR	745C	CMR	Clear R
CMY	745D	CMY	Clear Y
CMX	745E	CMX	Clear X
CMZ	745F	CMZ	Clear Z
CMF	7460	CMF	Clear F
CMH	7461	CMH	Clear H
CMR	7462	CMR	Clear R
CMY	7463	CMY	Clear Y
CMX	7464	CMX	Clear X
CMZ	7465	CMZ	Clear Z
CMF	7466	CMF	Clear F
CMH	7467	CMH	Clear H
CMR	7468	CMR	Clear R
CMY	7469	CMY	Clear Y
CMX	746A	CMX	Clear X
CMZ	746B	CMZ	Clear Z
CMF	746C	CMF	Clear F
CMH	746D	CMH	Clear H
CMR	746E	CMR	Clear R
CMY	746F	CMY	Clear Y
CMX	7470	CMX	Clear X
CMZ	7471	CMZ	Clear Z
CMF	7472	CMF	Clear F
CMH	7473	CMH	Clear H
CMR	7474	CMR	Clear R
CMY	7475	CMY	Clear Y
CMX	7476	CMX	Clear X
CMZ	7477	CMZ	Clear Z
CMF	7478	CMF	Clear F
CMH	7479	CMH	Clear H
CMR	747A	CMR	Clear R
CMY	747B	CMY	Clear Y
CMX	747C	CMX	Clear X
CMZ	747D	CMZ	Clear Z
CMF	747E	CMF	Clear F
CMH	747F	CMH	Clear H
CMR	7480	CMR	Clear R
CMY	7481	CMY	Clear Y
CMX	7482	CMX	Clear X
CMZ	7483	CMZ	Clear Z
CMF	7484	CMF	Clear F
CMH	7485	CMH	Clear H
CMR	7486	CMR	Clear R
CMY	7487	CMY	Clear Y
CMX	7488	CMX	Clear X
CMZ	7489	CMZ	Clear Z
CMF	748A	CMF	Clear F
CMH	748B	CMH	Clear H
CMR	748C	CMR	Clear R
CMY	748D	CMY	Clear Y
CMX	748E	CMX	Clear X
CMZ	748F	CMZ	Clear Z
CMF	7490	CMF	Clear F
CMH	7491	CMH	Clear H
CMR	7492	CMR	Clear R
CMY	7493	CMY	Clear Y
CMX	7494	CMX	Clear X
CMZ	7495	CMZ	Clear Z
CMF	7496	CMF	Clear F
CMH	7497	CMH	Clear H
CMR	7498	CMR	Clear R
CMY	7499	CMY	Clear Y
CMX	749A	CMX	Clear X
CMZ	749B	CMZ	Clear Z
CMF	749C	CMF	Clear F
CMH	749D	CMH	Clear H
CMR	749E	CMR	Clear R
CMY	749F	CMY	Clear Y
CMX	74A0	CMX	Clear X
CMZ	74A1	CMZ	Clear Z
CMF	74A2	CMF	Clear F
CMH	74A3	CMH	Clear H
CMR	74A4	CMR	Clear R
CMY	74A5	CMY	Clear Y
CMX	74A6	CMX	Clear X
CMZ	74A7	CMZ	Clear Z
CMF	74A8	CMF	Clear F
CMH	74A9	CMH	Clear H
CMR	74AA	CMR	Clear R
CMY	74AB	CMY	Clear Y
CMX	74AC	CMX	Clear X
CMZ	74AD	CMZ	Clear Z
CMF	74AE	CMF	Clear F
CMH	74AF	CMH	Clear H
CMR	74B0	CMR	Clear R
CMY	74B1	CMY	Clear Y
CMX	74B2	CMX	Clear X
CMZ	74B3	CMZ	Clear Z
CMF	74B4	CMF	Clear F
CMH	74B5	CMH	Clear H
CMR	74B6	CMR	Clear R
CMY	74B7	CMY	Clear Y
CMX	74B8	CMX	Clear X
CMZ	74B9	CMZ	Clear Z
CMF	74BA	CMF	Clear F
CMH	74BB	CMH	Clear H
CMR	74BC	CMR	Clear R
CMY	74BD	CMY	Clear Y
CMX	74BE	CMX	Clear X
CMZ	74BF	CMZ	Clear Z
CMF	74C0	CMF	Clear F
CMH	74C1	CMH	Clear H
CMR	74C2	CMR	Clear R
CMY	74C3	CMY	Clear Y
CMX	74C4	CMX	Clear X
CMZ	74C5	CMZ	Clear Z
CMF	74C6	CMF	Clear F
CMH	74C7	CMH	Clear H
CMR	74C8	CMR	Clear R
CMY	74C9	CMY	Clear Y
CMX	74CA	CMX	Clear X
CMZ	74CB	CMZ	Clear Z
CMF	74CC	CMF	Clear F
CMH	74CD	CMH	Clear H
CMR	74CE	CMR	Clear R
CMY	74CF	CMY	Clear Y
CMX	74D0	CMX	Clear X
CMZ	74D1	CMZ	Clear Z
CMF	74D2	CMF	Clear F
CMH	74D3	CMH	Clear H
CMR	74D4	CMR	Clear R
CMY	74D5	CMY	Clear Y
CMX	74D6	CMX	Clear X
CMZ	74D7	CMZ	Clear Z
CMF	74D8	CMF	Clear F
CMH	74D9	CMH	Clear H
CMR	74DA	CMR	Clear R
CMY	74DB	CMY	Clear Y
CMX	74DC	CMX	Clear X
CMZ	74DD	CMZ	Clear Z
CMF	74DE	CMF	Clear F
CMH	74DF	CMH	Clear H
CMR	74E0	CMR	Clear R
CMY	74E1	CMY	Clear Y
CMX	74E2	CMX	Clear X
CMZ	74E3	CMZ	Clear Z
CMF	74E4	CMF	Clear F
CMH	74E5	CMH	Clear H
CMR	74E6	CMR	Clear R
CMY	74E7	CMY	Clear Y
CMX	74E8	CMX	Clear X
CMZ	74E9	CMZ	Clear Z
CMF	74EA	CMF	Clear F
CMH	74EB	CMH	Clear H
CMR	74EC	CMR	Clear R
CMY	74ED	CMY	Clear Y
CMX	74EE	CMX	Clear X
CMZ	74EF	CMZ	Clear Z
CMF	74F0	CMF	Clear F
CMH	74F1	CMH	Clear H
CMR	74F2	CMR	Clear R
CMY	74F3	CMY	Clear Y
CMX	74F4	CMX	Clear X
CMZ	74F5	CMZ	Clear Z
CMF	74F6	CMF	Clear F
CMH	74F7	CMH	Clear H
CMR	74F8	CMR	Clear R
CMY	74F9	CMY	Clear Y
CMX	74FA	CMX	Clear X
CMZ	74FB	CMZ	Clear Z
CMF	74FC	CMF	Clear F
CMH	74FD	CMH	Clear H
CMR	74FE	CMR	Clear R
CMY	74FF	CMY	Clear Y



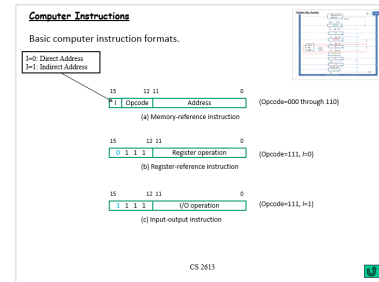
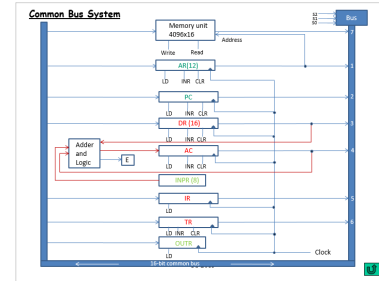
Timing Diagram



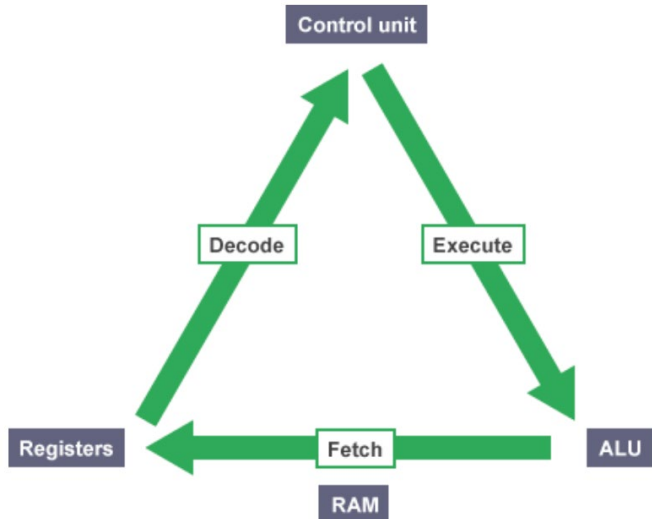
Important to understand the apparent delay in timing signals. A timing signal is associated with each rising edge. Thus, the statement **D3T4: SC $\leftarrow$ 0** is not performed at the beginning of the clock cycle where D3 and T4 are activated, but at the next rising edge

Instruction Cycle (four subcycles)

1. **Fetch** an instruction from memory
2. **Decode** the instruction
3. Read the effective **address** from memory (if indirect addressing is used)
4. Execute the instruction.



[Click for CPU Fetch-Decode-Execute Animation](#)



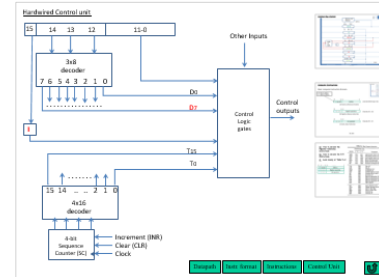
Q: How to decode the register-reference instructions.

Q: How to decode the I/O instructions.

A: Look closely at Table 5-2.

15	14	...	11	10	9	8	7	6	5	4	3	2	1	0
Instruction										Address				
1 0 0 0 1 1										Register operations				
1 1 1 1 1										I/O operations				

Symbol	Headword code	Description
AND	xxxx	AND memory word to AC
ASR	xxxx	ASR memory word to AC
LDN	xxxx	Load memory word to AC
STA	xxxx	Store content of AC to memory
BRN	xxxx	Branch unconditionally
BRA	xxxx	Branch and save return address
DEC	xxxx	Decrement and skip if zero
CLA	xxxx	Clear AC
CLE	xxxx	Clear E
CNA	xxxx	Complement AC
CME	xxxx	Complement E
CIR	xxxx	Complement right AC and E
CLL	xxxx	Complement left AC and E
INC	xxxx	Increment AC
INP	xxxx	Input character to AC
OUT	xxxx	Output character from AC
SKN	xxxx	Skip next instruction if AC is zero
SZL	xxxx	Skip next instruction if AC is less than zero
SZS	xxxx	Skip next instruction if E is 0
HLT	xxxx	HALT computer
INT	xxxx	Interrupt on
IOF	xxxx	Interrupt off



RTL specification for "fetch and load" phase:

This sequence is done for all instructions.

$$\begin{cases} T_0: AR \leftarrow PC \\ T_1: IR \leftarrow M[AR], PC \leftarrow PC+1 \\ T_2: D_0 \dots D_7 \leftarrow \text{Decode IR (12-14)}, AR \leftarrow IR (0-11), I \leftarrow IR (15) \end{cases}$$

$SC \leftarrow SC + 1$ assumed for every microop
 $SC \leftarrow 0$ shown explicitly.

Q: What happens on activation of timing signal T_3 ?

A: Depends on the status of D_7 and I .

If $D_7 = 0$, then a memory reference instruction is to be executed.

If $D_7 = 1$, then a register-reference or I/O instruction is to be executed.

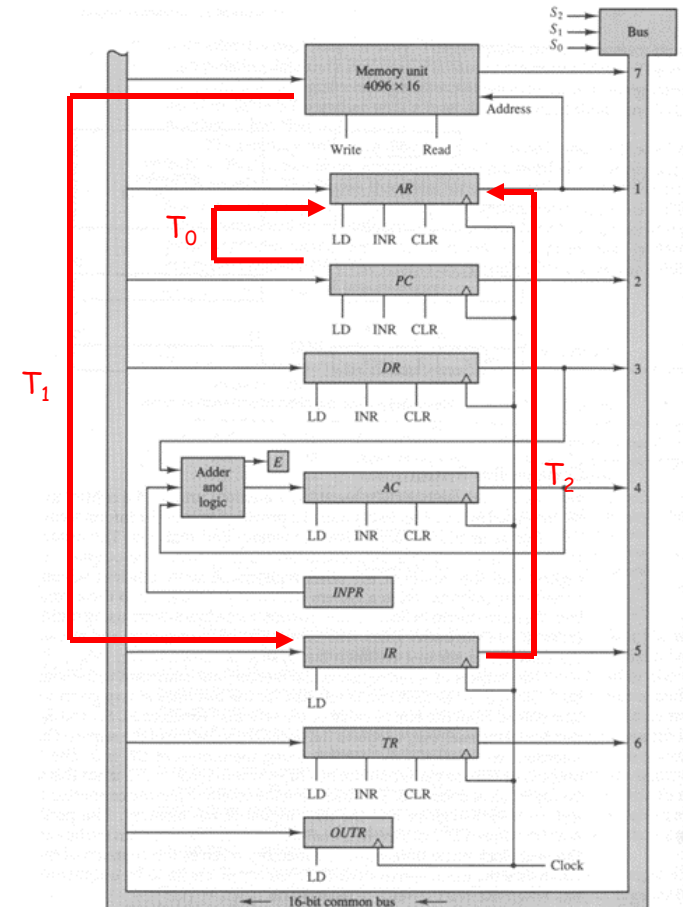
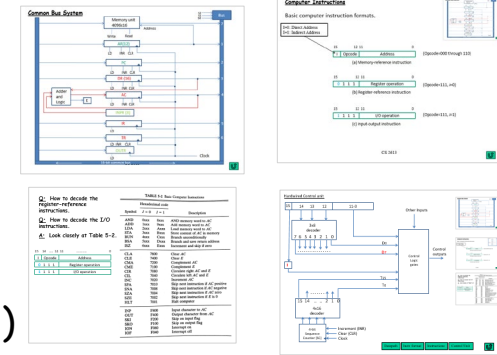
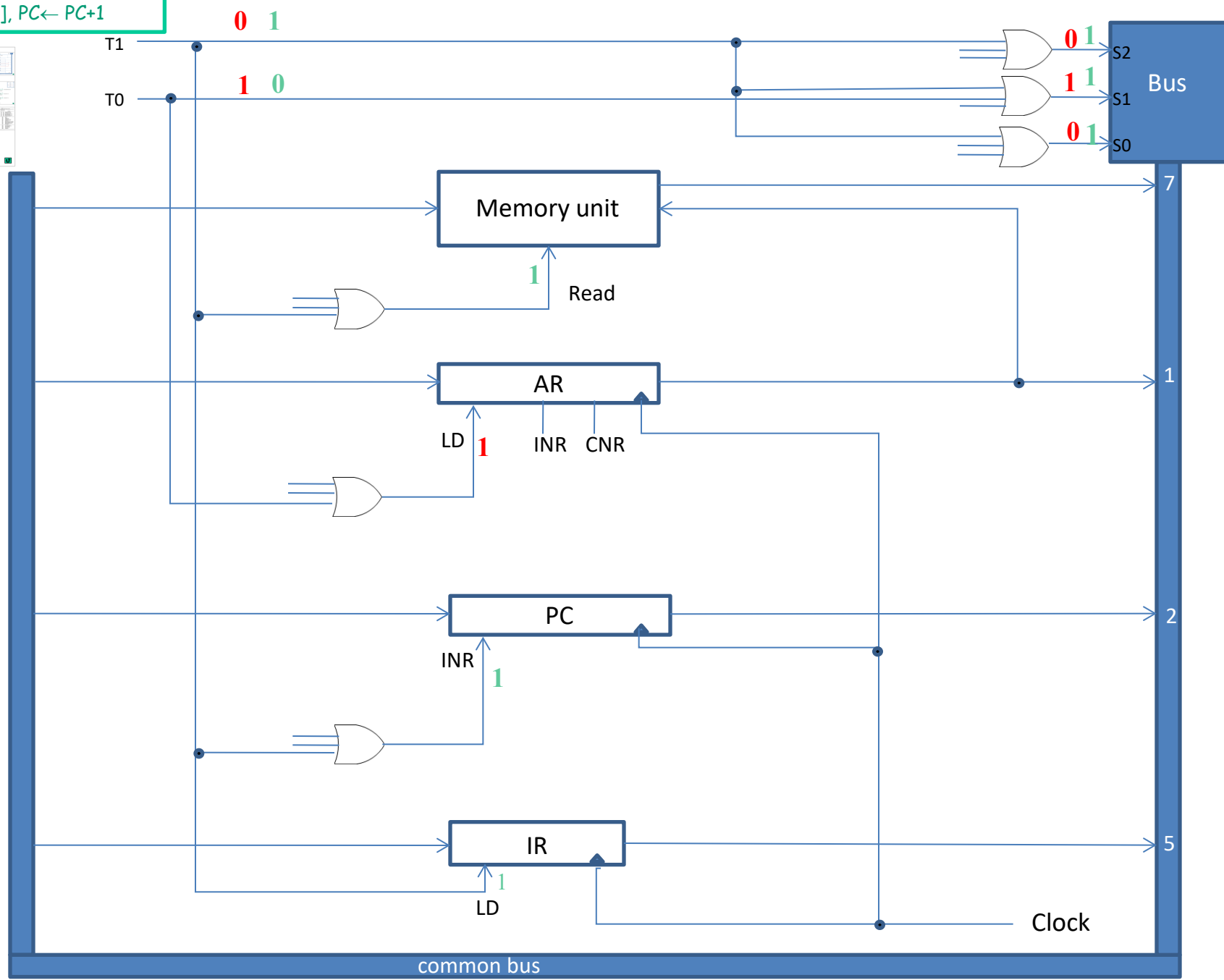
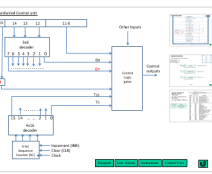


Figure 5-4 Basic computer registers connected to a common bus.

Datapath Instr. format Instructions Control Unit



Implementation of:
 $T_0: AR \leftarrow PC$
 $T_1: IR \leftarrow M[AR], PC \leftarrow PC+1$



Four specific cases

Nearly the same, but indirect addressing needs one extra cycle to get the effective address.

$D_7' I' \Rightarrow$ Memory-reference with direct addressing
 $D_7' I \Rightarrow$ Memory-reference with indirect addressing
 $D_7 I' \Rightarrow$ Register reference instruction
 $D_7 I \Rightarrow$ I/O instruction

$D_7' I' T_3$: Nothing (NOP)

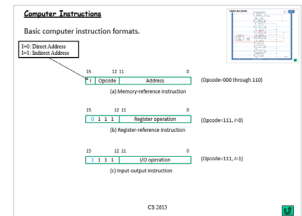
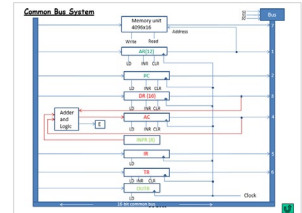
$D_7' I T_3$: $AR \leftarrow M[AR]$

(T_4 will be used to start executing memory reference instructions).

For other two cases, we can execute the appropriate instruction on T_3

$D_7 I' T_3$: Start executing register—ref. instr.

$D_7 I T_3$: Start executing I/O instr.

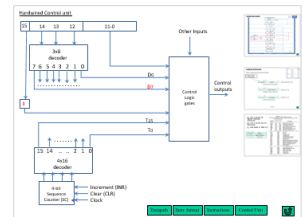


Q: How to decode the register-reference instructions.

Q: How to decode the I/O instructions.

d. Look closely at Table 5-2.

Instruction	Format	Description
AND	10 00 00 00	AND memory word to AC
OR	10 00 00 00	OR memory word to AC
LDA	10 00 00 00	Load memory word to AC
STA	10 00 00 00	Store memory word to AC
BRA	10 00 00 00	Branch and save return address
BRP	10 00 00 00	Branch and save return address
CLR	10 00 00 00	Clear AC
CLA	10 00 00 00	Clear AC
CMA	10 00 00 00	Complement AC
DMZ	10 00 00 00	Complement Z
ISZ	10 00 00 00	Increment PC and Z
DSZ	10 00 00 00	Decrement PC and Z
WAI	10 00 00 00	Wait for interrupt
HAL	10 00 00 00	Stop execution
IOI	10 00 00 00	Input/output instruction
OPR	10 00 00 00	Output operation
OPR	10 00 00 00	Output operation



Register Instructions: Operations during T_3

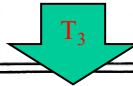


TABLE 5-3 Execution of Register-Reference Instructions

$D_7I'T_3 = r$ (common to all register-reference instructions)

$IR(i) = B_i$ [bit in $IR(0-11)$ that specifies the operation]

	r :	$SC \leftarrow 0$	Clear SC
CLA	rB_{11} :	$AC \leftarrow 0$	Clear AC
CLE	rB_{10} :	$E \leftarrow 0$	Clear E
CMA	rB_9 :	$AC \leftarrow \overline{AC}$	Complement AC
CME	rB_8 :	$E \leftarrow \overline{E}$	Complement E
CIR	rB_7 :	$AC \leftarrow shr AC, AC(15) \leftarrow E, E \leftarrow AC(0)$	Circulate right
CIL	rB_6 :	$AC \leftarrow shl AC, AC(0) \leftarrow E, E \leftarrow AC(15)$	Circulate left
INC	rB_5 :	$AC \leftarrow AC + 1$	Increment AC
SPA	rB_4 :	If $(AC(15) = 0)$ then $(PC \leftarrow PC + 1)$	Skip if positive
SNA	rB_3 :	If $(AC(15) = 1)$ then $(PC \leftarrow PC + 1)$	Skip if negative
SZA	rB_2 :	If $(AC = 0)$ then $PC \leftarrow PC + 1$	Skip if AC zero
SZE	rB_1 :	If $(E = 0)$ then $PC \leftarrow PC + 1$	Skip if E zero
HLT	rB_0 :	$S \leftarrow 0$ (S is a start-stop flip-flop)	Halt computer

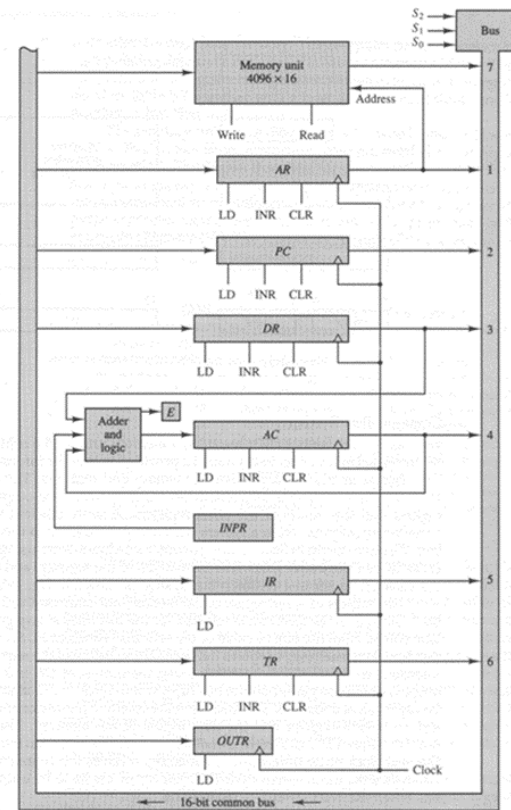


Figure 5-4 Basic computer registers connected to a common bus.

(Sequence Counter)

Note that the entry $r: SC \leftarrow 0$ is not actually an instruction, but occurs with every register-reference instruction.

Q: How could the start-stop flip flop be used to stop the computer?

A: Could be ANDed with the control input to the sequence counter increment (INR) line.

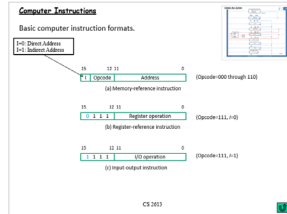
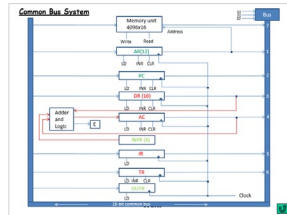
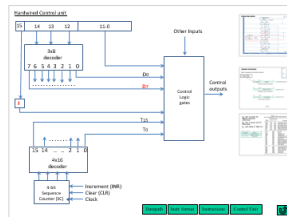


TABLE 5-3 Basic Computer Instructions

Instruction	Operation	Description
CLA	Clear AC	Clear AC
CLE	Clear E	Clear E
CMA	Complement AC	Complement AC
CME	Complement E	Complement E
CIR	Circulate right	Circulate right
CIL	Circulate left	Circulate left
INC	Increment AC	Increment AC
SPA	Skip if positive	Skip if $AC(15) = 0$
SNA	Skip if negative	Skip if $AC(15) = 1$
SZA	Skip if AC zero	Skip if $AC = 0$
SZE	Skip if E zero	Skip if $E = 0$
HLT	Halt computer	Halt computer



Alternate ways to specify "skip" instructions.

Assume AC register has a special "Z" bit to denote zero.

SPA $rB_4AC(15)'$: $PC \leftarrow PC + 1$
SNA $rB_3AC(15)$: $PC \leftarrow PC + 1$
SZA rB_2Z : $PC \leftarrow PC + 1$
SZE rB_1E : $PC \leftarrow PC + 1$

M-R Instructions: Operations during T_4 onwards

Note: Be careful when looking at Table 5-4 p. 145. Those are not technically RTL statements because they take more than one clock cycle.

AND

$D_0T_4: DR \leftarrow M[AR]$

$D_0T_5: AC \leftarrow AC \wedge DR, SC \leftarrow 0$

Note: There is an implied, " $SC \leftarrow SC + 1$ " here.

$T_0: AR \leftarrow PC$

$T_1: IR \leftarrow M[AR], PC \leftarrow PC + 1$

$T_2: D_0 \dots D_7 \leftarrow \text{Decode IR (12-14)}, AR \leftarrow \text{IR (0-11)}, I \leftarrow \text{IR (15)}$

$D_7' I' T_3: \text{Nothing (NOP)}$

$D_7' I T_3: AR \leftarrow M[AR]$

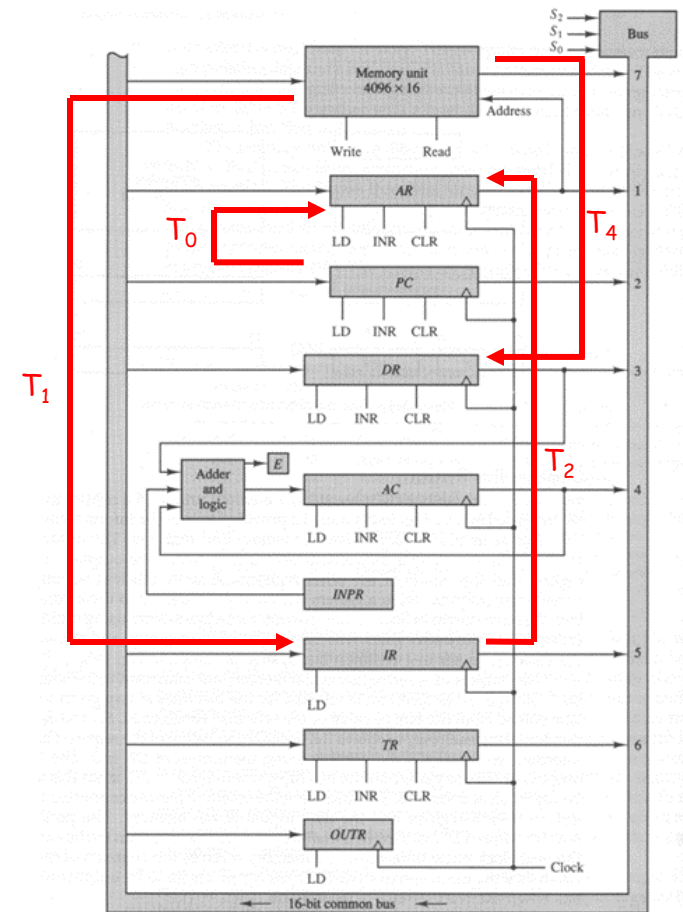
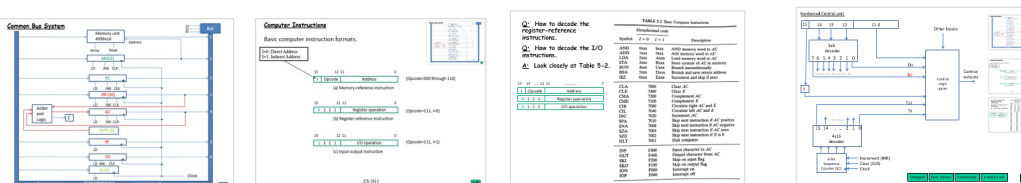


Figure 5-4 Basic computer registers connected to a common bus.



ADD

$D_1T_4: DR \leftarrow M[AR]$

$D_1T_5: AC \leftarrow AC + DR, E \leftarrow C_{out}, SC \leftarrow 0$

LDA (load to AC)

$D_2T_4: DR \leftarrow M[AR]$

$D_2T_5: AC \leftarrow DR, SC \leftarrow 0$

STA (store AC)

$D_3T_4: M[AR] \leftarrow AC, SC \leftarrow 0$

$T_0: AR \leftarrow PC$

$T_1: IR \leftarrow M[AR], PC \leftarrow PC + 1$

$T_2: D_0 \dots D_7 \leftarrow \text{Decode IR (12-14)}, AR \leftarrow IR (0-11), I \leftarrow IR (15)$

$D_7' I' T_3: \text{Nothing (NOP)}$

$D_7' I T_3: AR \leftarrow M[AR]$

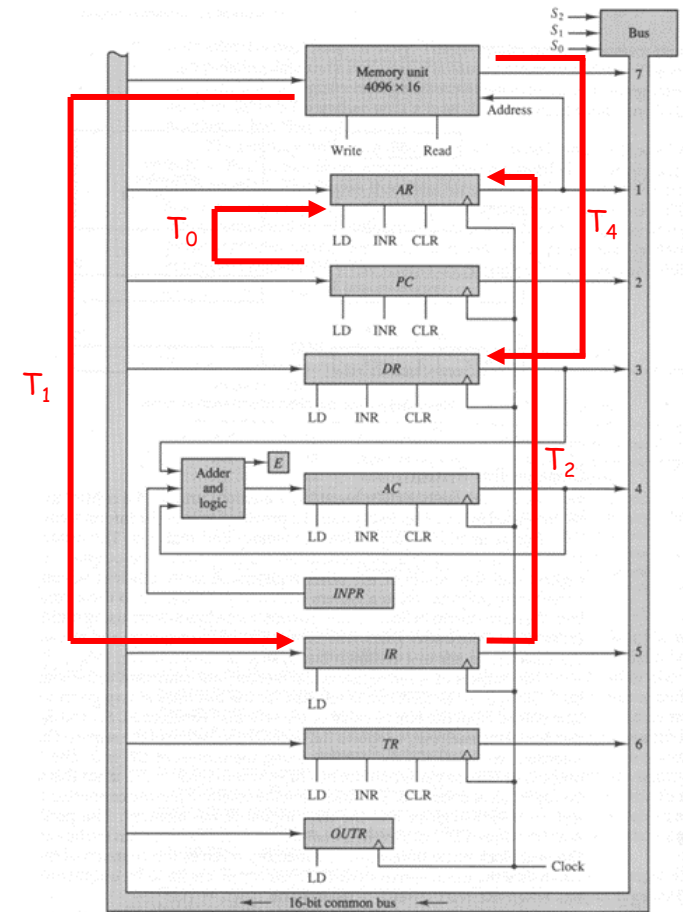
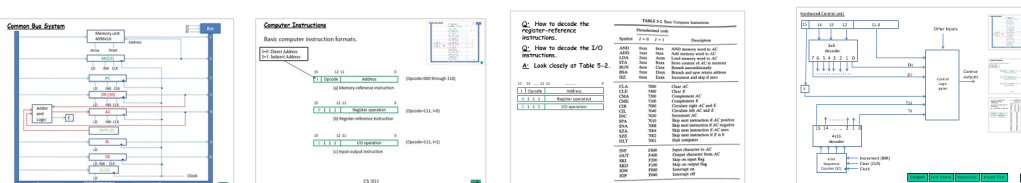


Figure 5-4 Basic computer registers connected to a common bus.



$T_0: AR \leftarrow PC$

$T_1: IR \leftarrow M[AR], PC \leftarrow PC + 1$

$T_2: D_0 \dots D_7 \leftarrow \text{Decode IR (12-14)}, AR \leftarrow IR (0-11), I \leftarrow IR (15)$

$D_7' I' T_3: \text{Nothing (NOP)}$

$D_7' I T_3: AR \leftarrow M[AR]$

BUN: (branch unconditionally)

Recall that indirect addressing could've been used.

$D_4 T_4: PC \leftarrow AR, \quad SC \leftarrow 0$

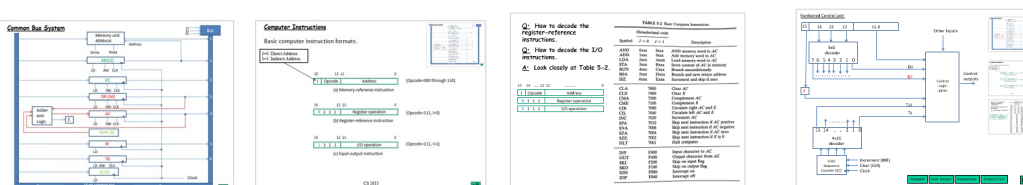
(Recall PC was already incremented at T_1 .)

BSA: (Branch and Save Return Address)

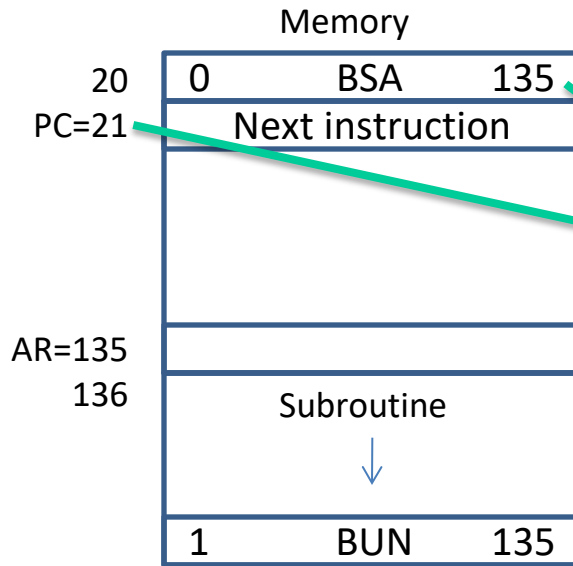
$D_5 T_4: M[AR] \leftarrow PC, \quad AR \leftarrow AR + 1$

$D_5 T_5: PC \leftarrow AR, \quad SC \leftarrow 0$

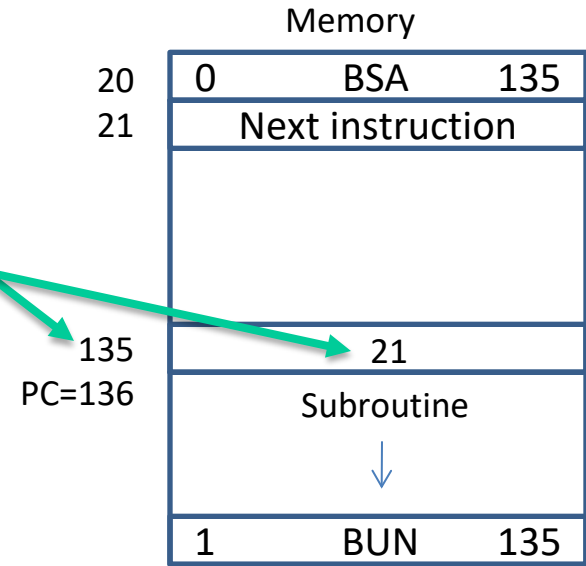
Similar to Function Call



Example of BSA instruction execution

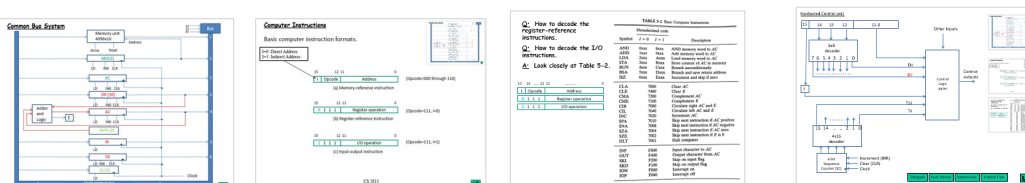


(a) Memory, PC, and AR at time T₄



(b) Memory and PC after execution

Note that I=0 in this example, but I=1 could also be used.



ISZ: (Increment and Skip if Zero)

$$T_0: AR \leftarrow PC$$

$T_1: IR \leftarrow M[AR], PC \leftarrow PC+1$

$$T_2: D_0 \dots D_7 \leftarrow \text{Decode IR (12-14)}, AR \leftarrow \text{IR (0-11)}, I \leftarrow \text{IR (15)}$$
$$D_6T_4: \quad DR \leftarrow M[AR]$$
$$D_6T_5: \quad DR \leftarrow DR + 1$$
$$D_6T_6: \quad M[AR] \leftarrow DR, \text{ if } (DR = 0) \text{ then } (PC \leftarrow PC + 1), SC \leftarrow 0$$

Can be used to implement a "for" loop.

Use a negative number (stored in memory) as the loop index. As the index is incremented, it will eventually equal to zero.

```
10 first instruction of loop body
```

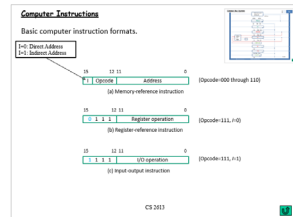
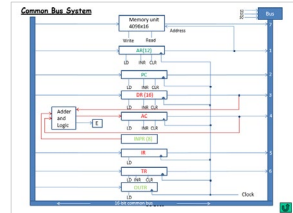
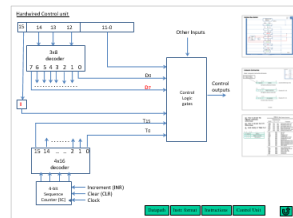
● ● ● ● ● ● ● ●

20 ISZ 23

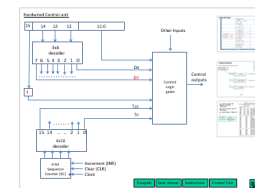
21 0 BUN 10

22 Next block of instructions

23 FFF0 ; Negative counter value

[illegible]

Memory-reference instructions



Interrupts

Note: In some applications, polling is necessary because there may not be other processing that can be done until input (or output) is performed.

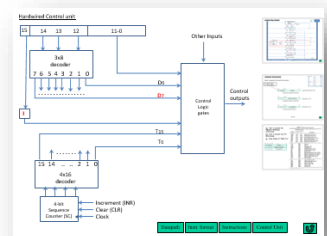
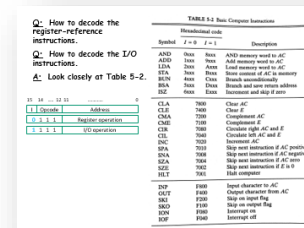
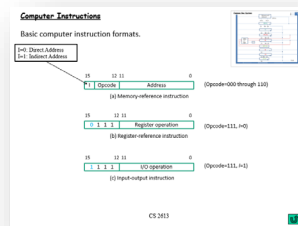
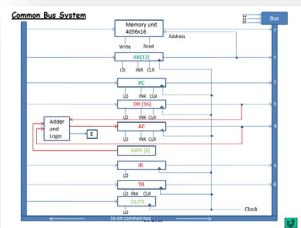
However, in some cases interrupt control is more efficient (the I/O device interrupts the computer to let it know a transfer is ready).

When the **IEN** flag is set to 1, then the flags **FGI** and **FGO** interrupt the computer.

Gives the programmer control of whether to enable the interrupt facility.

FLAGS

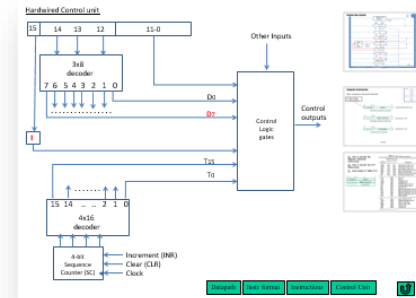
1. FGI
2. FGO
3. IEN



1. FGI
2. FGO
3. IEN
4. R

$$\underbrace{T'_0 \ T'_1 \ T'_2}_{1 \quad 1 \quad 1} (IEN)(FGI + FGO): R \leftarrow 1$$

A: Recall that FGI and FGO are set asynchronously by the I/O device. Thus, above allows R to be set anytime during the execute cycle of an instruction.



Interrupt Cycle

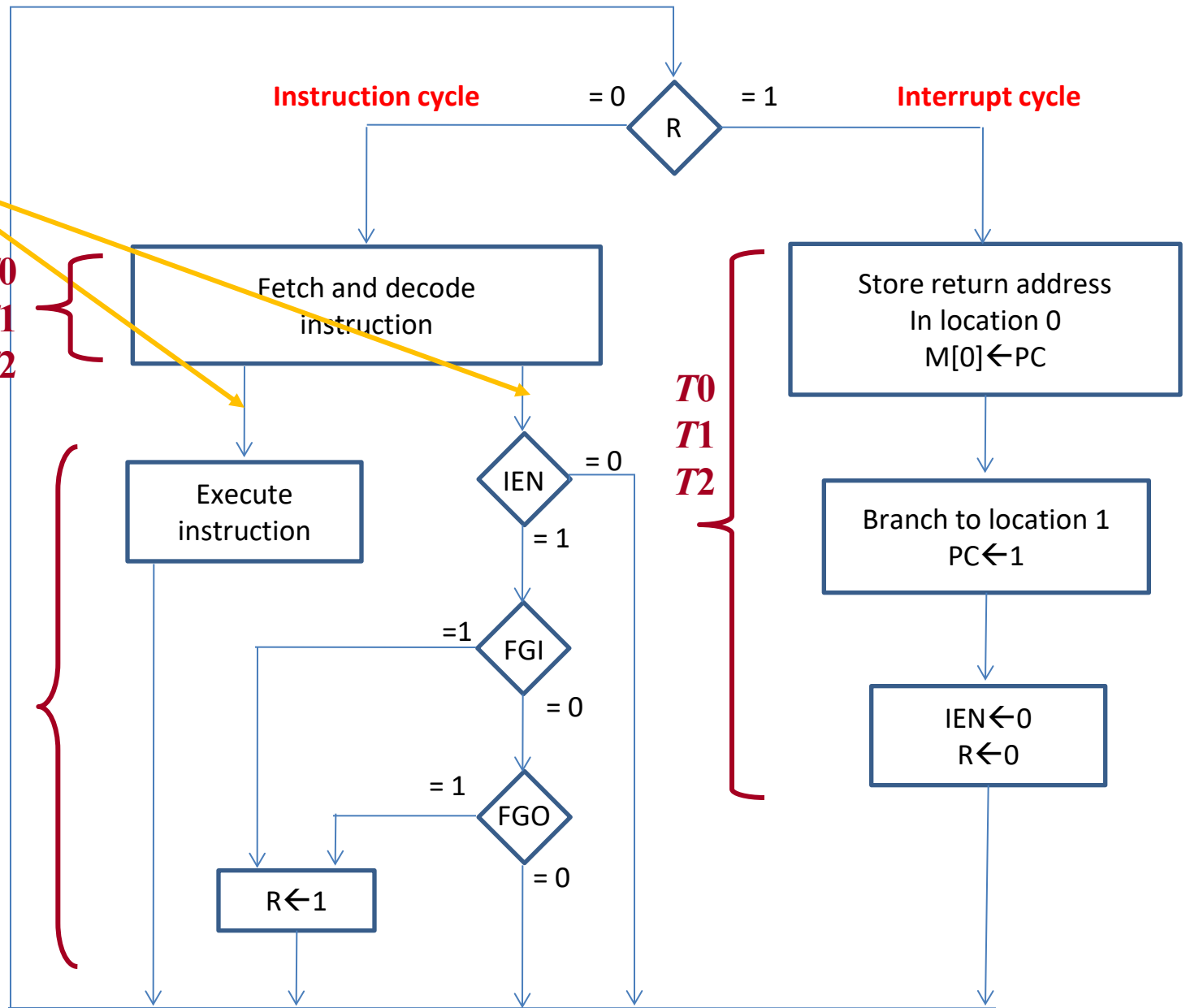
Both of these paths are activated at the same time (in parallel)

T0
T1
T2

T3
T4
...

Instruction cycle

Interrupt cycle



$T_0: AR \leftarrow PC$

$T_1: IR \leftarrow M[AR], PC \leftarrow PC + 1$

$T_2: D_0 \dots D_7 \leftarrow \text{Decode } IR(12-14), AR \leftarrow IR(0-11), I \leftarrow IR(15)$

$D'_7 I' T_3: \text{Nothing (NOP)}$

$D'_7 I T_3: AR \leftarrow M[AR]$

Control for fetch and decode microops must be modified...

*If $R = 0$
normal
instruction
cycle is
performed.*

Must
be added.

$R'T_0: AR \leftarrow PC$

$R'T_1: IR \leftarrow M[AR], PC \leftarrow PC + 1$

$R'T_2: D_0 \dots D_7 \leftarrow \text{Decode } IR(12-14), AR \leftarrow IR(0-11), I \leftarrow IR(15)$

*If $R = 1$, interrupt cycle is
performed, includes a
modified fetch phase*

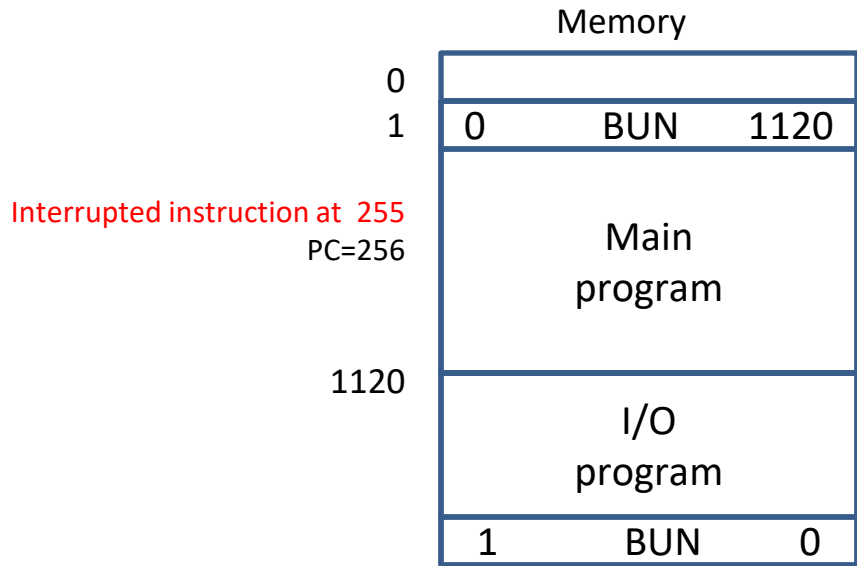
$RT_0: AR \leftarrow 0, TR \leftarrow PC$

$RT_1: M[AR] \leftarrow TR, PC \leftarrow 0$

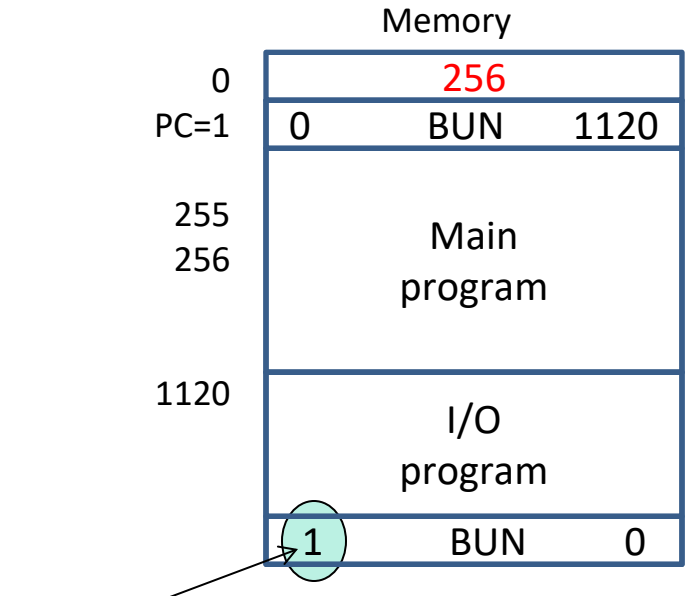
$RT_2: PC \leftarrow PC + 1, IEN \leftarrow 0, R \leftarrow 0, SC \leftarrow 0$

- Store PC in memory location 0
- Branch to location 1
- Clear IEN, R, and SC

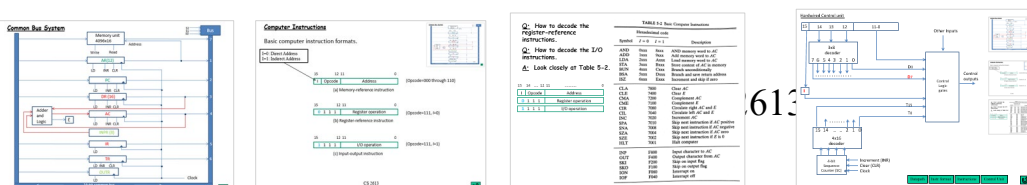
Demonstration of the interrupt cycle

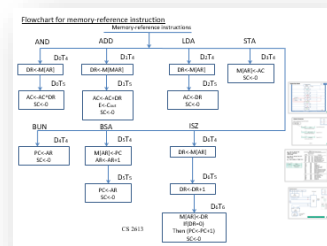


(a) Before interrupt



(b) after interrupt cycle



[illegible]

Design of Control Logic Gates (of Fig 5-16)

Inputs:

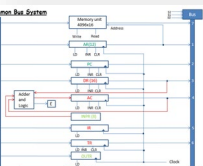
- 2 decoders
- 1 flip flop
- Bits 0-11 of IR
- Bits 0-15 of AC to check if AC=0 and sign (AC(15))
- Bits 0-15 of DR to check if DR=0
- 7 flip flops

Control Logic Gates

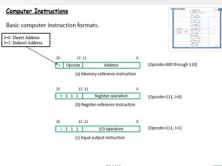
Outputs: Signals to control

- 9 registers
- R/W of memory
- Set, clear, complement FFs
- S₂, S₁, S₀ to select a register for bus
- AC adder and logic circuit

Common Bus System



Computer Extensions

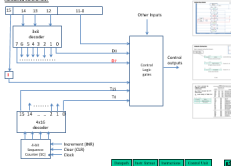


Q: How to decode the I/O instructions.

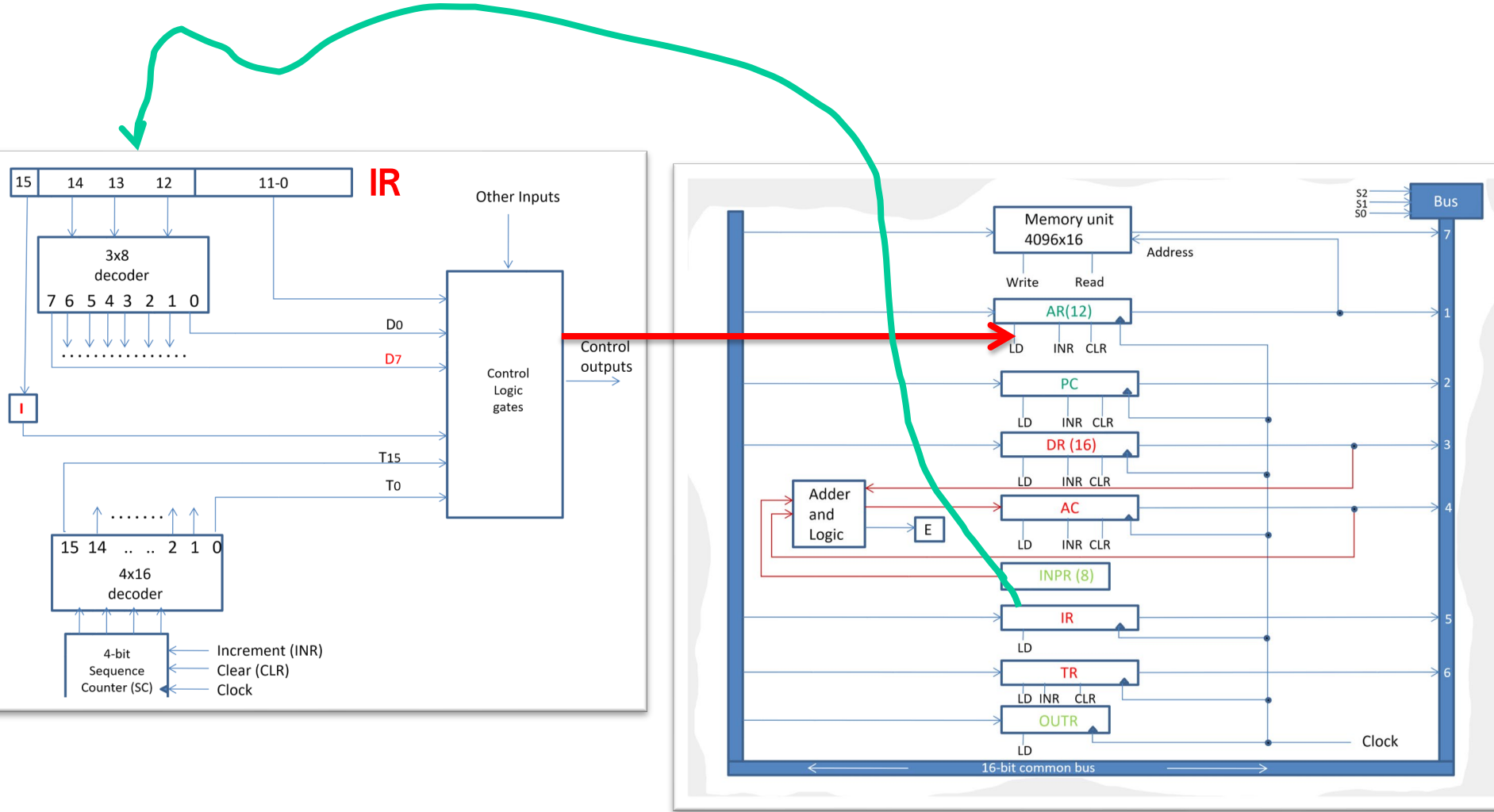
A: Look closely at Table 5-2

Instruction	Operation
ADD	Add contents of register to AC
AND	AND contents of register with AC
LD	Load contents of register to AC
ST	Store contents of AC to register
LDI	Load immediate value to AC
STI	Store immediate value to register
LDI	Load immediate value to AC
STI	Store immediate value to register

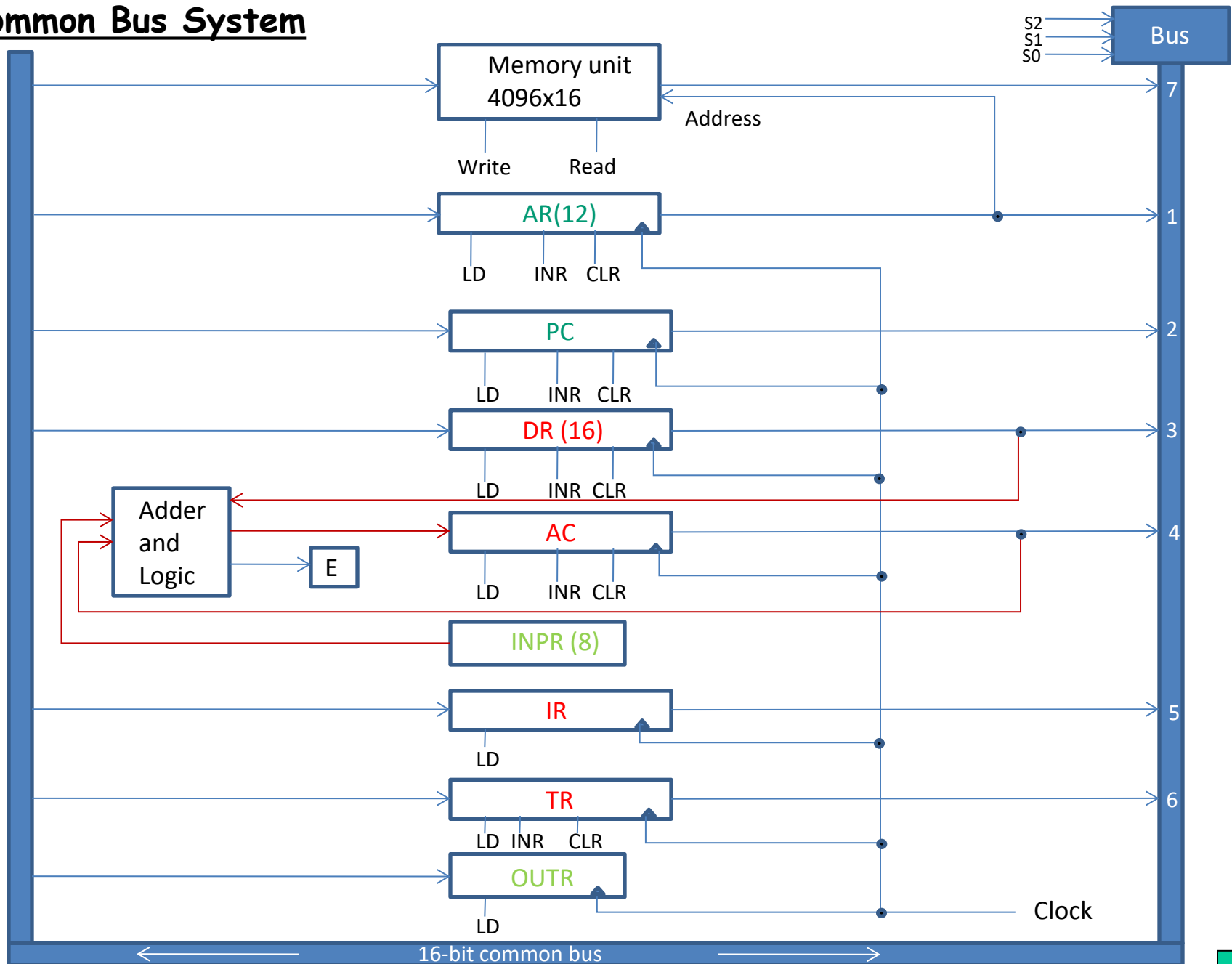
613



Connecting the Control Unit and Datapath



Common Bus System

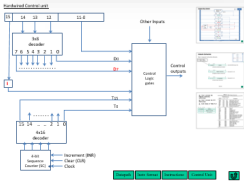
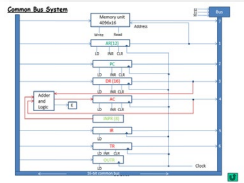


Control functions and microoperations for the basic computer

Microps
where AR is
changed

TABLE 5-6 Control Functions and Microoperations for the Basic Computer

Fetch	$R'T_0: AR \leftarrow PC$ $R'T_1: IR \leftarrow M[AR], PC \leftarrow PC + 1$
Decode	$R'T_2: D_0, \dots, D_7 \leftarrow \text{Decode } IR(12-14),$ $AR \leftarrow IR(0-11), I \leftarrow IR(15)$
Indirect	$D_4IT_3: AR \leftarrow M[AR]$
Interrupt:	$T_0T_1T_2(IEN)(FGI + FGO): R \leftarrow 1$ $RT_0: AR \leftarrow 0, TR \leftarrow PC$ $RT_1: M[AR] \leftarrow TR, PC \leftarrow 0$ $RT_2: PC \leftarrow PC + 1, IEN \leftarrow 0, R \leftarrow 0, SC \leftarrow 0$
Memory-reference:	
AND	$D_0T_4: DR \leftarrow M[AR]$ $D_0T_5: AC \leftarrow AC \wedge DR, SC \leftarrow 0$
ADD	$D_1T_4: DR \leftarrow M[AR]$ $D_1T_5: AC \leftarrow AC + DR, E \leftarrow C_{out}, SC \leftarrow 0$
LDA	$D_2T_4: DR \leftarrow M[AR]$ $D_2T_5: AC \leftarrow DR, SC \leftarrow 0$
STA	$D_3T_4: M[AR] \leftarrow AC, SC \leftarrow 0$
BUN	$D_4T_4: PC \leftarrow AR, SC \leftarrow 0$
BSA	$D_5T_4: M[AR] \leftarrow PC, AR \leftarrow AR + 1$ $D_5T_5: PC \leftarrow AR, SC \leftarrow 0$
ISZ	$D_6T_4: DR \leftarrow M[AR]$ $D_6T_5: DR \leftarrow DR + 1$ $D_6T_6: M[AR] \leftarrow DR, \text{ if } (DR = 0) \text{ then } (PC \leftarrow PC + 1), SC \leftarrow 0$
Register-reference:	
	$D_7IT_3 = r$ (common to all register-reference instructions) $IR(i) = B_i$ ($i = 0, 1, 2, \dots, 11$) $r: SC \leftarrow 0$ $rB_{11}: AC \leftarrow 0$ $rB_{10}: E \leftarrow 0$ $rB_9: AC \leftarrow \overline{AC}$ $rB_8: E \leftarrow \overline{E}$ $rB_7: AC \leftarrow \text{shr } AC, AC(15) \leftarrow E, E \leftarrow AC(0)$ $rB_6: AC \leftarrow \text{shl } AC, AC(0) \leftarrow E, E \leftarrow AC(15)$ $rB_5: AC \leftarrow AC + 1$ $rB_4: \text{If } (AC(15) = 0) \text{ then } (PC \leftarrow PC + 1)$ $rB_3: \text{If } (AC(15) = 1) \text{ then } (PC \leftarrow PC + 1)$ $rB_2: \text{If } (AC = 0) \text{ then } PC \leftarrow PC + 1$ $rB_1: \text{If } (E = 0) \text{ then } (PC \leftarrow PC + 1)$ $rB_0: S \leftarrow 0$
Input-output:	
	$D_7IT_3 = p$ (common to all input-output instructions) $IR(i) = B_i$ ($i = 6, 7, 8, 9, 10, 11$) $p: SC \leftarrow 0$ $pB_{11}: AC(0-7) \leftarrow \text{INPR}, FGI \leftarrow 0$ $pB_{10}: \text{OUTR} \leftarrow AC(0-7), FGO \leftarrow 0$ $pB_9: \text{If } (FGI = 1) \text{ then } (PC \leftarrow PC + 1)$ $pB_8: \text{If } (FGO = 1) \text{ then } (PC \leftarrow PC + 1)$ $pB_7: IEN \leftarrow 1$ $pB_6: IEN \leftarrow 0$



Control of Registers and Memory

Ex. Consider all microops for which the **content of AR** is changed.

$$R'T_0: AR \leftarrow PC$$

$$R'T_2: AR \leftarrow IR(0 - 11)$$

$$D_7'IT_3: AR \leftarrow M[AR]$$

$$RT_0: AR \leftarrow 0$$

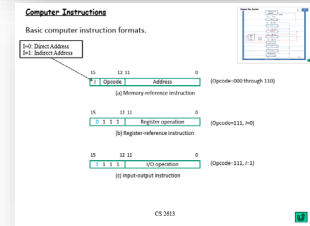
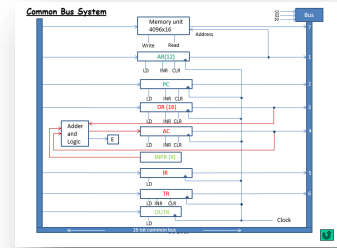
$$D_5T_4: AR \leftarrow AR + 1$$

So, the signals LD, CLR and INR (of AR) are controlled as follows:

$$LD(AR) = R'T_0 + R'T_2 + D_7'IT_3$$

$$CLR(AR) = RT_0$$

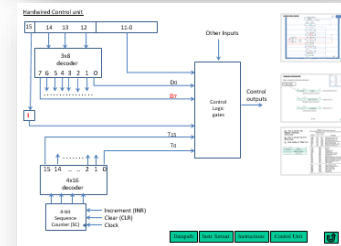
$$INR(AR) = D_5T_4$$



Q: How to decode the register-reference instructions.
Q: How to decode the I/O instructions.
A: Look closely at Table 5-2.

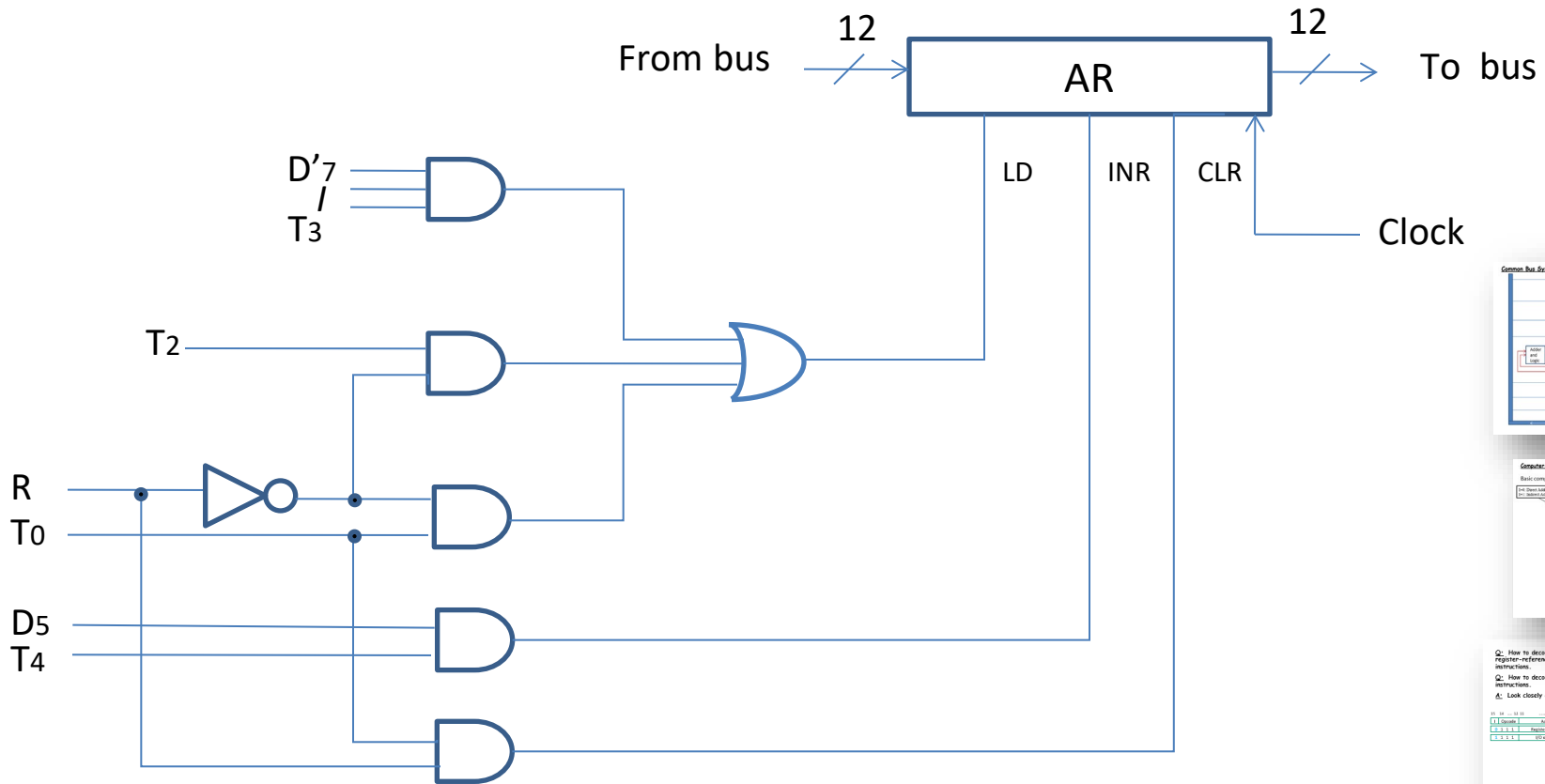
TABLE 5-2 Basic Computer Instructions

Symbol	$I = 0$	$I = 1$	Description
AND	0000	0001	AND memory word to AC
ADD	0010	0011	ADD memory word to AC
LDA	0100	0101	Load memory word to AC
STA	0110	0111	Store contents of AC to memory
BRA	1000	1001	Branch unconditionally
BRS	1010	1011	Branch on sign of E
INR	1100	1101	Increment and skip if zero
CLA	1110	1111	Clear AC
CLR	1100	1101	Clear E
CMA	1000	1001	Complement AC
CME	1010	1011	Complement E
CSN	0000	0001	Circular shift AC and E
CSL	0010	0011	Circular shift AC and E
INC	0100	0101	Increment AC
SPA	0110	0111	Skip next instruction if AC positive
SNA	1000	1001	Skip next instruction if AC negative
SZA	1010	1011	Skip next instruction if AC zero
SZC	1100	1101	Skip next instruction if E is 0
SZD	1110	1111	Skip next instruction if E is 1
STP	1100	1101	Input character to AC
OUT	1110	1111	Output character from AC
SKI	0000	0001	Skip on input flag
SNO	0010	0011	Skip on output flag
BNP	0100	0101	Branch on flag
BOP	0110	0111	Branch on flag



Similar logic design is performed for other registers and memory.

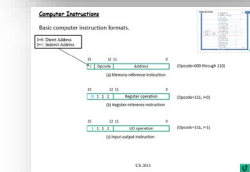
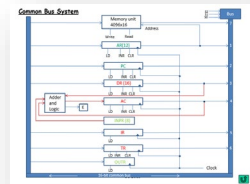
Control gates associated with AR



$$LD(AR) = R'T_0 + R'T_2 + D'_7IT_3$$

$$CLR(AR) = RT_0$$

$$INR(AR) = D_5T_4$$

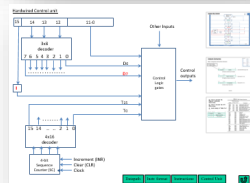


Q: How to decode the register-reference instructions.

Q: How to decode the I/O instructions.

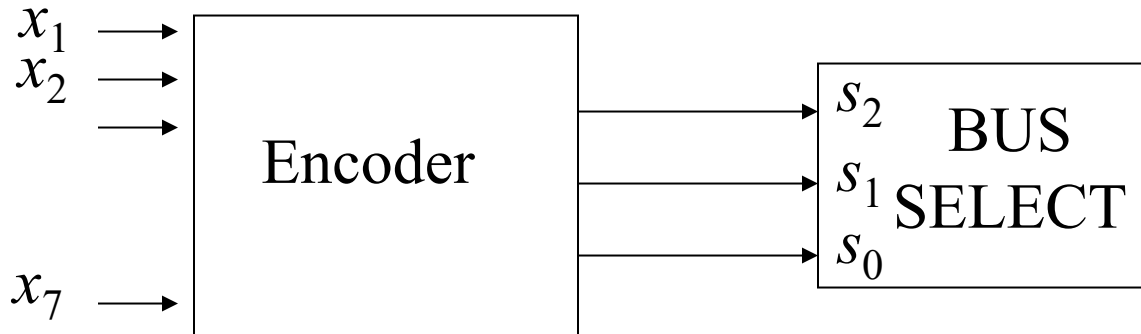
A: Look closely at Table 5-2.

Symbol	Q	R	I	Description
LD	0	0	0	Load register R with data from memory location Q.
INR	0	0	1	Increment register R.
CLR	0	0	1	Clear register R.
...	...	...	...	...



Control of the Bus

Let x_1 to x_7 denote signals for selection of registers and memory onto the bus, i.e., AR = 1, PC = 2, ... Mem unit = 7 (refer to Figure 5-4, next slide)

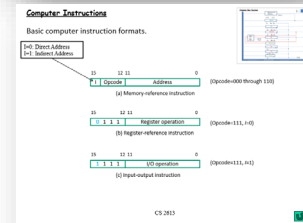
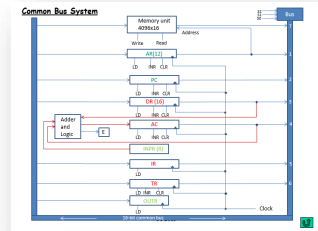
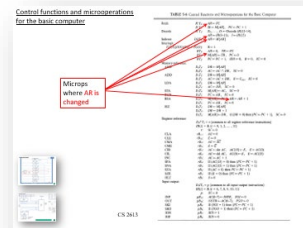


So, whenever AR is the source of microop, $x_1=1$.

$$D_4T_4: PC \leftarrow AR$$

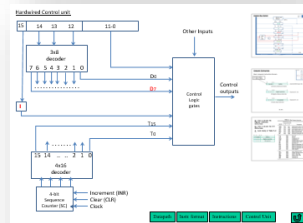
$$D_5T_5: PC \leftarrow AR$$

$$\text{Thus, } x_1 = D_4T_4 + D_5T_5$$



Q: How to decide the register-reference instructions.
Q: How to decide the I/O instructions.
A: Look closely at Table 5-2.

Symbol	IR	Description
AND	0000	AND memory word in AC
ADD	0001	ADD memory word in AC
STA	0010	Store memory word in AC
LDA	0011	Load memory word in AC
BRA	0100	Branch unconditionally
BNA	0101	Branch if not above
BNA	0110	Branch if not above
BNA	0111	Branch if not above
BNA	1000	Branch if not above
BNA	1001	Branch if not above
BNA	1010	Branch if not above
BNA	1011	Branch if not above
BNA	1100	Branch if not above
BNA	1101	Branch if not above
BNA	1110	Branch if not above
BNA	1111	Branch if not above



5-15. The memory unit of the basic computer shown in Fig. 5-3 is to be changed to a $65,536 \times 16$ memory, requiring an address of 16 bits. The instruction format of a memory-reference instruction shown in Fig. 5-5(a) remains the same for $I = 1$ (indirect address) with the address part of the instruction residing in positions 0 through 11. But when $I = 0$ (direct address), the address of the instruction is given by the 16 bits in the next word following the instruction. Modify the microoperations during time T_2 , T_3 , (and T_4 if necessary) to conform with this configuration.

$65,536 \times 16$ memory = 16 bits

